Mastering SQL, MySQL & Stored Procedures -- Architecture, Optimization, and Enterprise Usage Guide

---

Author: Senior Database Architect **Target Audience:** Java backend developers with 3+ years experience aiming for architect-level database expertise **Prerequisites:** Basic SQL knowledge, Java, Spring Boot fundamentals

---

Table of Contents

1. SQL Fundamentals
2. MySQL Architecture
3. SQL Query Mastery
4. Advanced SQL Queries
5. Database Design
6. Stored Procedures Deep Dive
7. Triggers
8. Indexing and Performance Optimization
9. Transactions and Concurrency Control
10. Query Optimization
11. Real Production Database Scenarios
12. MySQL + Spring Boot Integration
13. Stored Procedure Production Examples
14. Database Performance Tuning
15. Common Production Issues
16. Interview Questions (100+)
17. Hands-on SQL Exercises (30)

Part 1: SQL Fundamentals

1.1 What is SQL?

SQL (Structured Query Language) is the standard language for interacting with relational databases. Every backend system -- banking, e-commerce, insurance, government -- ultimately reads and writes data through SQL.

1.2 Types of SQL Commands

Category	Full Name	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Define/modify database structure
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Manipulate data
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions
DCL	Data Control Language	GRANT, REVOKE	Manage permissions

```sql
-- DDL: Create table
CREATE TABLE employees (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    department VARCHAR(50),
    salary DECIMAL(12,2),
    hire_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- DML: Insert data
INSERT INTO employees (name, email, department, salary, hire_date)
VALUES ('John Doe', 'john@company.com', 'Engineering', 95000.00, '2022-01-15');

-- TCL: Transaction
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
```

```sql
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;

-- DCL: Permissions
GRANT SELECT, INSERT ON employees TO 'app_user'@'localhost';
```

1.3 SQL Execution Lifecycle

```
Client sends SQL query
       |
       v
+-------------------------------+
| 1. CONNECTION HANDLER         |
|    Authenticate, assign thread|
+-------------------------------+
       |
       v
+-------------------------------+
| 2. PARSER                     |
|    Syntax check               |
|    Build parse tree            |
+-------------------------------+
       |
       v
+-------------------------------+
| 3. PREPROCESSOR               |
|    Validate table/column names|
|    Check permissions           |
+-------------------------------+
       |
       v
+-------------------------------+
| 4. QUERY OPTIMIZER            |
|    Generate execution plans   |
|    Choose best plan (cost)    |
|    Decide index usage         |
+-------------------------------+
       |
       v
+-------------------------------+
| 5. EXECUTION ENGINE           |
|    Execute chosen plan        |
|    Read/write data via        |
|    storage engine (InnoDB)    |
```

```
+-------------------------------+
       |
       v
+-------------------------------+
| 6. RESULT SET                 |
|    Format and return results  |
+-------------------------------+
```

1.4 Why SQL Matters for Backend Engineers

Scenario	SQL Impact
Slow API response	Usually a slow SQL query underneath
Data integrity	Constraints, transactions, ACID guarantee correctness
Reporting	Complex aggregations, window functions
Debugging production	Need to query data directly
Performance tuning	80% of backend performance is database performance
System design interviews	Database schema design is always tested

---

Part 2: MySQL Architecture

2.1 MySQL Server Architecture

```
+-----------------------------------------------------------------+
|                        MySQL Server                              |
|                                                                   |
|  +----------------------------+                                  |
|  |     Connection Layer       |  Thread per connection           |
|  |  (Authentication, SSL,     |  Connection pooling              |
|  |   Thread management)       |                                  |
|  +----------------------------+                                  |
|              |                                                    |
|              v                                                    |
|  +----------------------------+                                  |
|  |       SQL Layer            |                                  |
|  |  +--------+ +----------+  |                                 |
|  |  | Parser | | Optimizer|  |                                 |
|  |  +--------+ +----------+  |                                 |
|  |  +--------+ +----------+  |                                 |
|  |  | Cache  | | Executor |  |                                 |
|  |  +--------+ +----------+  |                                 |
|  +----------------------------+                                  |
|              |                                                    |
|              v                                                    |
|  +----------------------------+                                  |
|  |    Storage Engine Layer    |                                  |
|  |                            |                                  |
|  |  +--------+ +----------+  |                                 |
|  |  | InnoDB | | MyISAM   |  |                                 |
|  |  +--------+ +----------+  |                                 |
|  |  +--------+ +----------+  |                                 |
|  |  | Memory | | Archive  |  |                                 |
|  |  +--------+ +----------+  |                                 |
|  +----------------------------+                                  |
|              |                                                    |
|              v                                                    |
|  +----------------------------+                                  |
|  |     File System            |                                  |
|  |  Data files, Log files,    |                                  |
|  |  Index files               |                                  |
```

```
|  +---------------------------+                                  |
+-----------------------------------------------------------------+
```

2.2 InnoDB vs MyISAM

Feature	InnoDB	MyISAM
Transactions	Yes (ACID compliant)	No
Row-level locking	Yes	Table-level only
Foreign keys	Yes	No
Crash recovery	Yes (redo/undo logs)	No
MVCC	Yes	No
Full-text search	Yes (5.6+)	Yes
Storage	Clustered index (data in PK order)	Heap storage
Use case	OLTP, production systems	Read-heavy analytics, legacy

Rule: Always use InnoDB for production systems.

2.3 ACID Properties

Property	Meaning	MySQL Implementation
Atomicity	All or nothing	Undo log (rollback on failure)
Consistency	Valid state transitions	Constraints, triggers
Isolation	Concurrent transactions don't interfere	MVCC + isolation levels
Durability	Committed data survives crash	Redo log (write-ahead logging)

2.4 MVCC (Multi-Version Concurrency Control)

InnoDB maintains multiple versions of rows to allow readers and writers to operate concurrently without blocking each other.

```
Transaction T1 (READ):              Transaction T2 (WRITE):
SELECT * FROM accounts              UPDATE accounts
```

```
WHERE id = 1;                          SET balance = 500
                                       WHERE id = 1;
T1 sees the OLD version               T2 creates a NEW version
(consistent snapshot)                  (written to undo log)

Result: T1 reads without blocking T2
        T2 writes without blocking T1
```

2.5 InnoDB Locking

Lock Type	Scope	Use
Shared Lock (S)	Row	Allows concurrent reads, blocks writes
Exclusive Lock (X)	Row	Blocks all other access
Intention Shared (IS)	Table	Signals intent to lock rows for reading
Intention Exclusive (IX)	Table	Signals intent to lock rows for writing
Gap Lock	Between index values	Prevents phantom reads
Next-Key Lock	Row + gap before it	Default for REPEATABLE READ

```sql
-- Shared lock
SELECT * FROM accounts WHERE id = 1 LOCK IN SHARE MODE;

-- Exclusive lock
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
```

Part 3: SQL Query Mastery

3.1 SELECT

```sql
-- Basic select
SELECT name, email, salary FROM employees;

-- All columns (avoid in production!)
SELECT * FROM employees;  -- BAD: loads unnecessary data

-- Aliases
SELECT
    e.name AS employee_name,
    d.name AS department_name,
    e.salary * 12 AS annual_salary
FROM employees e
JOIN departments d ON e.department_id = d.id;
```

3.2 WHERE Clause

What: Filters rows based on conditions. **Internally:** Applied after FROM but before GROUP BY in execution order.

```sql
-- Comparison operators
SELECT * FROM employees WHERE salary > 80000;
SELECT * FROM employees WHERE department = 'Engineering';
SELECT * FROM employees WHERE hire_date >= '2023-01-01';

-- Logical operators
SELECT * FROM employees
WHERE department = 'Engineering' AND salary > 80000;

SELECT * FROM employees
WHERE department = 'Engineering' OR department = 'Product';

-- IN operator
SELECT * FROM employees
WHERE department IN ('Engineering', 'Product', 'Design');
```

```sql
-- BETWEEN
SELECT * FROM orders
WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31';

-- LIKE (pattern matching)
SELECT * FROM employees WHERE email LIKE '%@gmail.com';
SELECT * FROM employees WHERE name LIKE 'John%';

-- IS NULL
SELECT * FROM employees WHERE manager_id IS NULL; -- Top-level managers

-- NOT
SELECT * FROM employees WHERE department NOT IN ('HR', 'Legal');
```

3.3 GROUP BY and Aggregations

What: Groups rows by column values and applies aggregate functions.

```sql
-- Count employees per department
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;

-- Average salary per department
SELECT department,
       COUNT(*) AS count,
       AVG(salary) AS avg_salary,
       MIN(salary) AS min_salary,
       MAX(salary) AS max_salary,
       SUM(salary) AS total_payroll
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;

-- Monthly revenue
SELECT
    YEAR(order_date) AS year,
    MONTH(order_date) AS month,
    COUNT(*) AS total_orders,
    SUM(total_amount) AS revenue
FROM orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY year, month;
```

3.4 HAVING

What: Filters groups (applied AFTER GROUP BY, unlike WHERE which filters rows).

```sql
-- Departments with more than 10 employees
SELECT department, COUNT(*) AS count
FROM employees
GROUP BY department
HAVING COUNT(*) > 10;

-- High-spending customers
SELECT customer_id, SUM(total_amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(total_amount) > 10000
ORDER BY total_spent DESC;
```

3.5 ORDER BY

```sql
-- Single column sort
SELECT * FROM employees ORDER BY salary DESC;

-- Multiple columns
SELECT * FROM employees
ORDER BY department ASC, salary DESC;

-- Sort by expression
SELECT name, salary, salary * 12 AS annual
FROM employees
ORDER BY annual DESC;
```

3.6 LIMIT and OFFSET (Pagination)

```sql
-- First 10 results
SELECT * FROM products ORDER BY created_at DESC LIMIT 10;

-- Page 2 (items 11-20)
SELECT * FROM products ORDER BY created_at DESC LIMIT 10 OFFSET 10;
```

```sql
-- Page N: LIMIT page_size OFFSET (page_number * page_size)

-- Top 5 highest paid employees
SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 5;
```

3.7 SQL Execution Order

```
Written order:   SELECT -> FROM -> WHERE -> GROUP BY -> HAVING -> ORDER BY ->
LIMIT
Execution order: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY ->
LIMIT

This is why you can't use column aliases in WHERE but CAN in ORDER BY.
```

Part 4: Advanced SQL Queries

4.1 Subqueries

```sql
-- Scalar subquery (returns single value)
SELECT name, salary,
       (SELECT AVG(salary) FROM employees) AS company_avg
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- IN subquery
SELECT * FROM employees
WHERE department_id IN (
    SELECT id FROM departments WHERE location = 'New York'
);

-- EXISTS subquery (more efficient than IN for large datasets)
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.id
);
```

4.2 Correlated Subqueries

A subquery that references the outer query. Executes once per outer row.

```sql
-- Employees earning more than their department average
SELECT e.name, e.salary, e.department
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e.department  -- References outer query
);

-- Latest order per customer
SELECT * FROM orders o1
WHERE o1.created_at = (
    SELECT MAX(o2.created_at)
```

```
    FROM orders o2
    WHERE o2.customer_id = o1.customer_id
);
```

4.3 Joins

```sql
-- INNER JOIN (only matching rows from both tables)
SELECT e.name, d.name AS department
FROM employees e
INNER JOIN departments d ON e.department_id = d.id;

-- LEFT JOIN (all rows from left, matching from right, NULL if no match)
SELECT c.name, COUNT(o.id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
GROUP BY c.id, c.name;
-- Shows ALL customers, even those with zero orders

-- RIGHT JOIN (all rows from right, matching from left)
SELECT e.name, d.name AS department
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.id;
-- Shows ALL departments, even those with no employees

-- FULL OUTER JOIN (MySQL workaround using UNION)
SELECT e.name, d.name AS department
FROM employees e LEFT JOIN departments d ON e.department_id = d.id
UNION
SELECT e.name, d.name AS department
FROM employees e RIGHT JOIN departments d ON e.department_id = d.id;

-- Self Join (table joined with itself)
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

4.4 Window Functions

Window functions perform calculations across a set of rows related to the current row WITHOUT collapsing rows (unlike GROUP BY).

```sql
-- ROW_NUMBER: Assign sequential numbers
SELECT
    name, department, salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;

-- RANK and DENSE_RANK
SELECT
    name, department, salary,
    RANK() OVER (ORDER BY salary DESC) AS rank_with_gaps,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS rank_no_gaps
FROM employees;

-- Partition by department (rank within each department)
SELECT
    name, department, salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS
dept_rank
FROM employees;

-- Running total
SELECT
    order_date,
    total_amount,
    SUM(total_amount) OVER (ORDER BY order_date) AS running_total
FROM orders;

-- Moving average (last 7 days)
SELECT
    order_date,
    daily_revenue,
    AVG(daily_revenue) OVER (
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS moving_avg_7d
FROM daily_sales;

-- LAG and LEAD (previous/next row values)
SELECT
    month,
    revenue,
    LAG(revenue, 1) OVER (ORDER BY month) AS prev_month,
    revenue - LAG(revenue, 1) OVER (ORDER BY month) AS growth
FROM monthly_revenue;

-- NTILE (divide into buckets)
```

```sql
SELECT
    name, salary,
    NTILE(4) OVER (ORDER BY salary) AS salary_quartile
FROM employees;
```

4.5 Common Table Expressions (CTE)

```sql
-- Basic CTE
WITH high_earners AS (
    SELECT name, department, salary
    FROM employees
    WHERE salary > 100000
)
SELECT department, COUNT(*) AS count, AVG(salary) AS avg_salary
FROM high_earners
GROUP BY department;

-- Multiple CTEs
WITH
    dept_stats AS (
        SELECT department, AVG(salary) AS avg_salary, COUNT(*) AS emp_count
        FROM employees
        GROUP BY department
    ),
    high_salary_depts AS (
        SELECT department, avg_salary
        FROM dept_stats
        WHERE avg_salary > 80000
    )
SELECT * FROM high_salary_depts ORDER BY avg_salary DESC;

-- Recursive CTE (organizational hierarchy)
WITH RECURSIVE org_chart AS (
    -- Anchor: top-level managers (no manager)
    SELECT id, name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive: employees reporting to previous level
    SELECT e.id, e.name, e.manager_id, oc.level + 1
    FROM employees e
    INNER JOIN org_chart oc ON e.manager_id = oc.id
```

```sql
)
SELECT * FROM org_chart ORDER BY level, name;
```

Part 5: Database Design

5.1 Normalization

First Normal Form (1NF)

- Each column contains atomic (indivisible) values
- No repeating groups

```
-- BAD (not 1NF): phone_numbers contains multiple values
-- | id | name | phone_numbers          |
-- | 1  | John | 555-1234, 555-5678     |

-- GOOD (1NF): separate table for phones
CREATE TABLE employees (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100)
);

CREATE TABLE employee_phones (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    employee_id BIGINT NOT NULL,
    phone VARCHAR(20) NOT NULL,
    FOREIGN KEY (employee_id) REFERENCES employees(id)
);
```

Second Normal Form (2NF)

- Must be in 1NF
- All non-key columns depend on the ENTIRE primary key (no partial dependencies)

Third Normal Form (3NF)

- Must be in 2NF
- No transitive dependencies (non-key column depends on another non-key column)

```
-- BAD (not 3NF): department_name depends on department_id, not employee_id
-- | employee_id | name | department_id | department_name |

-- GOOD (3NF): separate departments table
CREATE TABLE departments (
```

```sql
    id BIGINT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE employees (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100),
    department_id BIGINT,
    FOREIGN KEY (department_id) REFERENCES departments(id)
);
```

5.2 Denormalization

Intentionally adding redundancy to improve read performance.

When to Normalize	When to Denormalize
Write-heavy OLTP systems	Read-heavy reporting/analytics
Data integrity is critical	Query performance is critical
Storage is a concern	Query simplicity matters
Few complex queries	Many complex JOINs

```sql
-- Normalized: requires JOIN for every order query
SELECT o.*, c.name, c.email FROM orders o JOIN customers c ON o.customer_id =
c.id;

-- Denormalized: customer name stored in orders table (redundant but fast)
SELECT order_id, customer_name, total_amount FROM orders;
-- No JOIN needed! But must keep customer_name in sync
```

5.3 Database Design for Microservices

```
Rule 1: Each service owns its database
Rule 2: No cross-service foreign keys
Rule 3: Communication via APIs or events

+------------+    +------------+    +------------+
| User       |    | Order      |    | Payment    |
```

```
| Service    |    | Service    |    | Service    |
+-----+------+    +-----+------+    +-----+------+
      |                 |                 |
+-----+------+    +-----+------+    +-----+------+
| user_db    |    | order_db   |    | payment_db |
|            |    |            |    |            |
| users      |    | orders     |    | payments   |
| addresses  |    | order_items|    | refunds    |
| user_prefs |    | (stores    |    | txn_log    |
|            |    | customer_id|    |            |
|            |    | NOT FK)    |    |            |
+------------+    +------------+    +------------+
```

5.4 Production Schema Example: E-Commerce

```sql
CREATE TABLE customers (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    phone VARCHAR(20),
    status ENUM('ACTIVE','SUSPENDED','DELETED') DEFAULT 'ACTIVE',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_email (email),
    INDEX idx_status (status)
);

CREATE TABLE products (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    sku VARCHAR(50) UNIQUE NOT NULL,
    category VARCHAR(100),
    price DECIMAL(12,2) NOT NULL,
    stock_quantity INT NOT NULL DEFAULT 0,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_category (category),
    INDEX idx_sku (sku)
);

CREATE TABLE orders (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    customer_id BIGINT NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
```

```sql
    status ENUM('PENDING','CONFIRMED','SHIPPED','DELIVERED','CANCELLED')
DEFAULT 'PENDING',
    shipping_address TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_customer (customer_id),
    INDEX idx_status (status),
    INDEX idx_created (created_at),
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);

CREATE TABLE order_items (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,
    line_total DECIMAL(12,2) GENERATED ALWAYS AS (quantity * unit_price)
STORED,
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id),
    INDEX idx_order (order_id)
);
```

Part 6: Stored Procedures Deep Dive

6.1 What Are Stored Procedures?

A stored procedure is a **precompiled set of SQL statements** stored in the database and executed as a single unit. Think of it as a function in the database.

6.2 Why Use Stored Procedures?

Benefit	Explanation
Performance	Precompiled execution plan, no repeated parsing
Reduced network calls	Multiple SQL statements in one call
Security	Grant EXECUTE permission without direct table access
Encapsulation	Business logic centralized in database
Reusability	Call from any application (Java, Python, etc.)
Atomicity	All statements in one transaction

6.3 When to Use in Production

Use Case	Example
Complex business calculations	Interest calculation in banking
Batch data processing	Monthly statement generation
Data validation	Eligibility checks in government systems
Report generation	Aggregate and format report data
Data migration	Transform and move data between tables

6.4 Basic Syntax

```sql
-- Create a simple procedure
DELIMITER //
CREATE PROCEDURE GetEmployeesByDepartment(
    IN dept_name VARCHAR(50)
)
BEGIN
    SELECT id, name, email, salary
    FROM employees
    WHERE department = dept_name
    ORDER BY salary DESC;
END //
DELIMITER ;

-- Call the procedure
CALL GetEmployeesByDepartment('Engineering');
```

6.5 Parameter Types

```sql
DELIMITER //
CREATE PROCEDURE TransferFunds(
    IN from_account_id BIGINT,      -- Input only
    IN to_account_id BIGINT,        -- Input only
    IN amount DECIMAL(12,2),        -- Input only
    OUT transfer_id BIGINT,         -- Output only
    OUT status_message VARCHAR(100) -- Output only
)
BEGIN
    DECLARE from_balance DECIMAL(12,2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET status_message = 'Transfer failed due to database error';
        SET transfer_id = -1;
    END;

    START TRANSACTION;

    -- Check balance
    SELECT balance INTO from_balance
    FROM accounts WHERE id = from_account_id FOR UPDATE;

    IF from_balance < amount THEN
```

```
        SET status_message = 'Insufficient funds';
        SET transfer_id = -1;
        ROLLBACK;
    ELSE
        -- Debit
        UPDATE accounts SET balance = balance - amount WHERE id =
from_account_id;
        -- Credit
        UPDATE accounts SET balance = balance + amount WHERE id =
to_account_id;
        -- Log
        INSERT INTO transactions (from_account, to_account, amount, type,
created_at)
        VALUES (from_account_id, to_account_id, amount, 'TRANSFER', NOW());

        SET transfer_id = LAST_INSERT_ID();
        SET status_message = 'Transfer successful';
        COMMIT;
    END IF;
END //
DELIMITER ;

-- Call with OUT parameters
CALL TransferFunds(1, 2, 500.00, @tid, @msg);
SELECT @tid AS transfer_id, @msg AS status;
```

6.6 Control Flow

```
DELIMITER //
CREATE PROCEDURE CalculateDiscount(
    IN customer_id BIGINT,
    OUT discount_pct DECIMAL(5,2)
)
BEGIN
    DECLARE total_spent DECIMAL(12,2);
    DECLARE membership_years INT;

    -- Get customer data
    SELECT SUM(total_amount), TIMESTAMPDIFF(YEAR, MIN(created_at), NOW())
    INTO total_spent, membership_years
    FROM orders
    WHERE customer_id = customer_id AND status = 'DELIVERED';

    -- IF-ELSEIF-ELSE
```

```
    IF total_spent > 50000 AND membership_years > 5 THEN
        SET discount_pct = 20.00;
    ELSEIF total_spent > 25000 OR membership_years > 3 THEN
        SET discount_pct = 15.00;
    ELSEIF total_spent > 10000 THEN
        SET discount_pct = 10.00;
    ELSE
        SET discount_pct = 5.00;
    END IF;
END //
DELIMITER ;
```

6.7 Cursors (Row-by-Row Processing)

```
DELIMITER //
CREATE PROCEDURE GenerateMonthlyStatements(
    IN statement_month INT,
    IN statement_year INT
)
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE cust_id BIGINT;
    DECLARE cust_email VARCHAR(150);
    DECLARE total DECIMAL(12,2);

    DECLARE customer_cursor CURSOR FOR
        SELECT c.id, c.email, COALESCE(SUM(o.total_amount), 0)
        FROM customers c
        LEFT JOIN orders o ON c.id = o.customer_id
            AND MONTH(o.created_at) = statement_month
            AND YEAR(o.created_at) = statement_year
        WHERE c.status = 'ACTIVE'
        GROUP BY c.id, c.email;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN customer_cursor;

    read_loop: LOOP
        FETCH customer_cursor INTO cust_id, cust_email, total;
        IF done THEN LEAVE read_loop; END IF;

        INSERT INTO monthly_statements
            (customer_id, statement_month, statement_year, total_amount,
```

```
generated_at)
        VALUES
            (cust_id, statement_month, statement_year, total, NOW());
    END LOOP;

    CLOSE customer_cursor;
END //
DELIMITER ;
```

Part 7: Triggers

7.1 What Are Triggers?

A trigger is a stored SQL program that **automatically executes** when a specific event (INSERT, UPDATE, DELETE) occurs on a table. Triggers fire without explicit calls.

Why Triggers Exist

Triggers enforce **database-level business rules** that cannot be bypassed -- regardless of whether data is modified by application code, direct SQL, batch jobs, or admin tools. They guarantee rules are applied consistently.

When to Use Triggers

- **Audit logging** -- automatically recording who changed what and when
- **Data validation** -- enforcing constraints that CHECK constraints cannot express
- **Derived data maintenance** -- auto-updating stock counts, balances, denormalized columns
- **Soft delete enforcement** -- preventing actual DELETE operations

When NOT to Use Triggers

- **Complex business logic** -- put it in the application layer where it's testable and debuggable
- **Cross-table cascading updates** -- hard to debug, causes hidden performance issues
- **Notification/messaging** -- use application events or Kafka instead

> ***Production Warning:*** *Triggers are invisible to application developers. Over-reliance on triggers makes the system hard to debug because side effects happen silently. Always document triggers prominently.*

7.2 Trigger Types

Trigger	When It Fires	Common Use
`BEFORE INSERT`	Before a new row is inserted	Validate/modify data before saving
`AFTER INSERT`	After a new row is inserted	Audit logging, sync denormalized data
`BEFORE UPDATE`	Before a row is updated	Validate changes, preserve history
`AFTER UPDATE`	After a row is updated	Log changes, notify systems

`BEFORE DELETE`	Before a row is deleted	Prevent deletion, archive data
`AFTER DELETE`	After a row is deleted	Cleanup related data, audit

7.3 Audit Logging Trigger

```sql
-- Audit log table
CREATE TABLE audit_log (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    table_name VARCHAR(100) NOT NULL,
    record_id BIGINT NOT NULL,
    action VARCHAR(10) NOT NULL,
    old_values JSON,
    new_values JSON,
    changed_by VARCHAR(100),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Trigger: log every change to employees table
DELIMITER //
CREATE TRIGGER trg_employees_after_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (table_name, record_id, action, old_values,
new_values, changed_by)
    VALUES (
        'employees',
        NEW.id,
        'UPDATE',
        JSON_OBJECT(
            'name', OLD.name,
            'salary', OLD.salary,
            'department', OLD.department,
            'is_active', OLD.is_active
        ),
        JSON_OBJECT(
            'name', NEW.name,
            'salary', NEW.salary,
            'department', NEW.department,
            'is_active', NEW.is_active
        ),
        CURRENT_USER()
    );
```

```
END //
DELIMITER ;
```

7.4 Data Validation Trigger

```
DELIMITER //
CREATE TRIGGER trg_orders_before_insert
BEFORE INSERT ON orders
FOR EACH ROW
BEGIN
    -- Validate: amount must be positive
    IF NEW.total_amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Order amount must be greater than zero';
    END IF;

    -- Auto-set created timestamp
    SET NEW.created_at = NOW();

    -- Default status
    IF NEW.status IS NULL THEN
        SET NEW.status = 'PENDING';
    END IF;
END //
DELIMITER ;
```

7.5 Automatic Stock Update Trigger

```
-- After order item is inserted, decrease product stock
DELIMITER //
CREATE TRIGGER trg_order_items_after_insert
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    UPDATE products
    SET stock_quantity = stock_quantity - NEW.quantity
    WHERE id = NEW.product_id;

    -- Check if stock went negative
    IF (SELECT stock_quantity FROM products WHERE id = NEW.product_id) < 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient stock for product';
    END IF;
END //
DELIMITER ;

-- After order is cancelled, restore stock
DELIMITER //
CREATE TRIGGER trg_orders_after_update_status
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    IF OLD.status != 'CANCELLED' AND NEW.status = 'CANCELLED' THEN
        UPDATE products p
        INNER JOIN order_items oi ON p.id = oi.product_id
        SET p.stock_quantity = p.stock_quantity + oi.quantity
        WHERE oi.order_id = NEW.id;
    END IF;
END //
DELIMITER ;
```

7.6 Soft Delete Trigger

```
-- Prevent actual DELETE; mark as deleted instead
DELIMITER //
CREATE TRIGGER trg_customers_before_delete
BEFORE DELETE ON customers
FOR EACH ROW
BEGIN
    -- Instead of deleting, update status
    UPDATE customers
    SET status = 'DELETED', updated_at = NOW()
    WHERE id = OLD.id;

    -- Prevent the actual delete
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Use soft delete: UPDATE status to DELETED instead';
END //
DELIMITER ;
```

Part 8: Indexing and Performance Optimization

Why Indexing is the Most Important Database Concept

Indexing is the **single most impactful technique** for database performance. A missing index can make a query **1,000x slower**. Understanding indexing is what separates junior from senior database engineers.

How Indexing Works -- The Book Analogy

```
A database index works exactly like a book's index:

Without index:  Read every page to find "transactions" -> 5 minutes
With index:     Look up "transactions" in index -> page 247 -> 5 seconds

A database index stores sorted key values with pointers to actual rows,
allowing the engine to jump directly to matching rows instead of scanning all
rows.
```

Design Pattern: B+ Tree

MySQL InnoDB indexes use **B+ Trees** -- balanced tree structures where:

- Leaf nodes contain actual data or pointers to data
- Internal nodes contain keys for navigation
- All leaf nodes are linked for range scans
- Tree height is typically 3-4 levels, so any lookup takes 3-4 disk reads

8.1 What Is an Index?

An index is a **data structure** (usually B+ tree) that speeds up data retrieval. Like a book's index that tells you which page a topic is on, a database index tells MySQL which row a value is in.

```
WITHOUT Index:
  SELECT * FROM employees WHERE email = 'john@test.com';
  MySQL scans ALL 1,000,000 rows (full table scan)
  Time: 2000ms

WITH Index on email:
  MySQL walks the B+ tree directly to the row
  Time: 1ms
```

8.2 Index Types

Clustered Index (Primary Key)

```
- InnoDB tables ALWAYS have a clustered index
- Data rows are physically stored in primary key order
- One per table (the primary key)
- Finding by PK is the fastest possible query

+--------+--------+--------+--------+
| id=1   | id=2   | id=3   | id=4   |  Data stored in PK order
| row    | row    | row    | row    |
+--------+--------+--------+--------+
```

Non-Clustered Index (Secondary)

```
-- Points to the primary key, not directly to data
CREATE INDEX idx_email ON employees(email);
-- Lookup: idx_email -> primary key -> data row (two lookups)

+--------+--------+--------+--------+
| alice@ | bob@   | carl@  | dana@  |  Index sorted by email
| -> PK3 | -> PK1 | -> PK7 | -> PK2 |  Points to primary keys
+--------+--------+--------+--------+
```

Composite Index

```
CREATE INDEX idx_dept_salary ON employees(department, salary);

-- USES INDEX (leftmost prefix rule):
SELECT * FROM employees WHERE department = 'Engineering';              -- Yes
SELECT * FROM employees WHERE department = 'Eng' AND salary > 80000;  -- Yes
SELECT * FROM employees WHERE department = 'Eng' ORDER BY salary;     -- Yes

-- DOES NOT USE INDEX:
SELECT * FROM employees WHERE salary > 80000;  -- No! Leftmost column not used
```

Covering Index

```sql
CREATE INDEX idx_covering ON employees(department, salary, name);

-- This query is answered ENTIRELY from the index (no table lookup):
SELECT name, salary FROM employees WHERE department = 'Engineering';
-- "Covering" because all SELECT columns are in the index
```

8.3 When to Use Indexes

Use Index	Don't Use Index
Columns in WHERE clauses	Tables with < 1000 rows
Columns in JOIN conditions	Columns with very low cardinality (e.g., boolean)
Columns in ORDER BY	Columns that are frequently updated
Columns in GROUP BY	Tables that are write-heavy with few reads
Unique constraints	Wide columns (TEXT, BLOB)

8.4 Index Performance Example

```sql
-- Table: orders (5 million rows)
-- Query: Find recent orders for a customer

-- WITHOUT INDEX: Full table scan
EXPLAIN SELECT * FROM orders
WHERE customer_id = 42 AND status = 'PENDING'
ORDER BY created_at DESC LIMIT 10;
-- type: ALL, rows: 5,000,000, time: 3200ms

-- ADD INDEX
CREATE INDEX idx_customer_status_date
ON orders(customer_id, status, created_at);

-- WITH INDEX: Index lookup
EXPLAIN SELECT * FROM orders
WHERE customer_id = 42 AND status = 'PENDING'
ORDER BY created_at DESC LIMIT 10;
-- type: ref, rows: 15, time: 1ms
```

8.5 Index Anti-Patterns

```sql
-- 1. Function on indexed column (index NOT used)
SELECT * FROM employees WHERE YEAR(hire_date) = 2024;  -- BAD
SELECT * FROM employees WHERE hire_date >= '2024-01-01'
                          AND hire_date < '2025-01-01'; -- GOOD

-- 2. LIKE with leading wildcard (index NOT used)
SELECT * FROM employees WHERE name LIKE '%John%';  -- BAD (full scan)
SELECT * FROM employees WHERE name LIKE 'John%';   -- GOOD (uses index)

-- 3. OR with non-indexed column
SELECT * FROM employees WHERE email = 'x@y.com' OR phone = '555-1234';
-- If only email is indexed, full scan may be needed

-- 4. Implicit type conversion
SELECT * FROM employees WHERE id = '42';  -- String compared to INT
-- May prevent index usage depending on engine
```

Part 9: Transactions and Concurrency Control

What is a Transaction?

A **transaction** is a group of SQL statements that are treated as a **single unit of work**. Either ALL statements succeed (COMMIT) or ALL are undone (ROLLBACK). There is no partial execution.

Why Transactions Matter

Without transactions, a failure between two related operations leaves the database in an **inconsistent state**:

```
Transfer $500 from Account A to Account B:

Without transaction:                With transaction:
  1. Debit A: -$500  ✓               1. START TRANSACTION
  2. ⚡ APPLICATION CRASHES ⚡          2. Debit A: -$500
  3. Credit B: never happens!        3. Credit B: +$500
  Result: $500 disappeared!          4. COMMIT (or ROLLBACK if error)
                                     Result: Either both or neither
```

ACID Properties

Property	Meaning	Guarantee
Atomicity	All or nothing	Transaction fully completes or fully rolls back
Consistency	Valid state only	Database moves from one valid state to another
Isolation	No interference	Concurrent transactions don't affect each other
Durability	Permanent	Committed data survives crashes

Step-by-Step: Transaction Lifecycle

```
1. START TRANSACTION  -> Begin a new unit of work
2. Execute SQL statements (INSERT, UPDATE, DELETE)
3. If all succeed:  COMMIT   -> Changes written to disk permanently
4. If any fails:    ROLLBACK -> All changes undone, as if nothing happened
```

9.1 Transaction Syntax

```sql
-- Transfer $500 from account 1 to account 2
START TRANSACTION;

    UPDATE accounts SET balance = balance - 500 WHERE id = 1;
    UPDATE accounts SET balance = balance + 500 WHERE id = 2;
    INSERT INTO transactions (from_id, to_id, amount, type)
    VALUES (1, 2, 500, 'TRANSFER');

COMMIT;  -- All changes saved permanently

-- If anything fails:
ROLLBACK;  -- ALL changes undone (atomicity)
```

9.2 Savepoints

```sql
START TRANSACTION;

    INSERT INTO orders (customer_id, total_amount) VALUES (1, 100);
    SAVEPOINT after_order;

    INSERT INTO order_items (order_id, product_id, quantity) VALUES (1, 5, 2);

    -- Oops, wrong product
    ROLLBACK TO SAVEPOINT after_order;
    -- Only the order_items insert is undone, order is still there

    INSERT INTO order_items (order_id, product_id, quantity) VALUES (1, 8, 2);

COMMIT;
```

9.3 Isolation Levels

What Are Isolation Levels?

Isolation levels define **how much one transaction can see another transaction's uncommitted changes**. Higher isolation = more consistency but lower performance (more locking).

Why Multiple Isolation Levels Exist

Full isolation (SERIALIZABLE) is the safest but slowest. Most applications don't need it. Isolation levels let you **trade consistency for performance** based on your use case:

```
Less Isolation                                      More Isolation
More Performance                                    Less Performance
+----------------++--------------++---------------++--------------+
|READ UNCOMMITTED||READ COMMITTED||REPEATABLE READ|| SERIALIZABLE |
+----------------++--------------++---------------++--------------+
   Never use            Oracle/PG         MySQL default        Financial
   in production        default                               systems only
```

When to Use Each Level

Level	Use When
READ COMMITTED	Most web apps; good balance of performance and consistency
REPEATABLE READ	Default in MySQL; prevents most anomalies
SERIALIZABLE	Financial transactions where absolute consistency is required

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
READ UNCOMMITTED	Yes	Yes	Yes	Fastest
READ COMMITTED	No	Yes	Yes	Good
REPEATABLE READ (MySQL default)	No	No	Nearly No*	Medium
SERIALIZABLE	No	No	No	Slowest

*InnoDB uses gap locks to prevent most phantom reads even at REPEATABLE READ.

Dirty Read Example

```sql
-- READ UNCOMMITTED allows reading uncommitted data from other transactions

-- Transaction 1:
START TRANSACTION;
UPDATE accounts SET balance = 1000 WHERE id = 1;
```

```
-- NOT committed yet!

-- Transaction 2 (READ UNCOMMITTED):
SELECT balance FROM accounts WHERE id = 1;
-- Returns 1000 (uncommitted data!)

-- Transaction 1:
ROLLBACK;
-- Transaction 2 read data that never actually existed!
```

Non-Repeatable Read Example

```
-- Transaction 1 (READ COMMITTED):
START TRANSACTION;
SELECT balance FROM accounts WHERE id = 1;  -- Returns 500

-- Transaction 2 commits an update:
UPDATE accounts SET balance = 800 WHERE id = 1;
COMMIT;

-- Transaction 1 reads again:
SELECT balance FROM accounts WHERE id = 1;  -- Returns 800 (different!)
-- Same query, different result within same transaction
```

```
-- Set isolation level
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
-- Or for global:
SET GLOBAL TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

9.4 Deadlocks

Two transactions waiting for each other's locks.

```
-- Transaction 1:                    Transaction 2:
START TRANSACTION;                   START TRANSACTION;
UPDATE accounts                      UPDATE accounts
SET balance = 100                    SET balance = 200
WHERE id = 1;   -- Locks row 1      WHERE id = 2;   -- Locks row 2
```

```
UPDATE accounts                    UPDATE accounts
SET balance = 200                  SET balance = 100
WHERE id = 2;   -- WAITS for T2   WHERE id = 1;   -- WAITS for T1

-- DEADLOCK! MySQL detects it and kills one transaction
```

How to prevent deadlocks:

1. Always access tables/rows in the same order
2. Keep transactions short
3. Use appropriate isolation level
4. Add proper indexes to reduce lock scope

Part 10: Query Optimization

Why Query Optimization Matters

In production, a single unoptimized query can bring down an entire application. Understanding the EXPLAIN output and optimization techniques is essential for any backend engineer.

The Optimization Process

```
Step 1: Identify slow queries (slow query log or APM tools)
Step 2: Run EXPLAIN on the query
Step 3: Analyze the execution plan (scan type, rows, indexes)
Step 4: Apply optimization (add index, rewrite query, restructure)
Step 5: Re-run EXPLAIN to verify improvement
```

10.1 EXPLAIN Command

```sql
EXPLAIN SELECT o.id, c.name, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.status = 'PENDING'
ORDER BY o.created_at DESC
LIMIT 10;
```

EXPLAIN Output Columns

Column	Meaning	What to Look For
type	Join type	ALL is bad; ref, eq_ref, const are good
rows	Estimated rows scanned	Lower is better
key	Index actually used	NULL means no index used
Extra	Additional info	"Using filesort" and "Using temporary" are concerns

Join Types (best to worst)

Type	Speed	Description
`const`	Fastest	Primary key or unique index lookup
`eq_ref`	Fast	Unique index in JOIN (one row per match)
`ref`	Good	Non-unique index lookup
`range`	OK	Index range scan (BETWEEN, >, <)
`index`	Slow	Full index scan (reads entire index)
`ALL`	Slowest	Full table scan (reads every row)

10.2 Optimization Techniques

1. Add Missing Indexes

```sql
-- Before: Full table scan
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
-- type: ALL, rows: 5000000

-- Fix:
CREATE INDEX idx_customer_id ON orders(customer_id);

-- After: Index lookup
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
-- type: ref, rows: 25
```

2. Use Covering Index

```sql
-- Before: Index lookup + table lookup (two steps)
SELECT customer_id, status, total_amount FROM orders WHERE customer_id = 42;

-- Create covering index
CREATE INDEX idx_covering ON orders(customer_id, status, total_amount);
-- After: Index-only scan (one step, no table lookup)
-- Extra: Using index
```

3. Avoid SELECT *

```sql
-- BAD: Loads all columns from disk
SELECT * FROM orders WHERE customer_id = 42;

-- GOOD: Load only needed columns
SELECT id, status, total_amount FROM orders WHERE customer_id = 42;
```

4. Optimize Subqueries with JOIN

```sql
-- SLOW: Correlated subquery runs once per row
SELECT * FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.department =
e.department);

-- FASTER: JOIN with precomputed averages
SELECT e.*
FROM employees e
INNER JOIN (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
) dept_avg ON e.department = dept_avg.department
WHERE e.salary > dept_avg.avg_salary;
```

5. Optimize Pagination for Deep Pages

```sql
-- SLOW: OFFSET skips rows one by one
SELECT * FROM orders ORDER BY id LIMIT 10 OFFSET 1000000;
-- Scans 1,000,010 rows!

-- FAST: Keyset pagination (seek method)
SELECT * FROM orders
WHERE id > 1000000  -- Start from last seen ID
ORDER BY id
LIMIT 10;
-- Scans only 10 rows using index
```

6. Use EXISTS Instead of IN for Large Datasets

```sql
-- SLOW for large subquery results:
SELECT * FROM customers WHERE id IN (SELECT customer_id FROM orders);

-- FASTER:
SELECT * FROM customers c WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.id
);
```

Part 11: Real Production Database Scenarios

Why Production SQL Differs from Textbook SQL

Production database code must handle:

- **Concurrent access** -- multiple users hitting the same data simultaneously
- **Failure recovery** -- transactions, savepoints, error handlers
- **Performance at scale** -- millions of rows, complex joins, indexes
- **Data integrity** -- foreign keys, constraints, FOR UPDATE locks

The following stored procedures demonstrate how these concerns are handled in real enterprise systems.

11.1 Order Processing System

Concept: Atomic Multi-Table Operations

Placing an order involves multiple tables (orders, order_items, products). All must succeed or all must fail. The stored procedure uses `FOR UPDATE` to lock product rows and prevent overselling.

```
-- Place an order (transaction ensures atomicity)
DELIMITER //
CREATE PROCEDURE PlaceOrder(
    IN p_customer_id BIGINT,
    IN p_product_ids JSON,
    IN p_quantities JSON,
    IN p_shipping_address TEXT,
    OUT p_order_id BIGINT,
    OUT p_status VARCHAR(50)
)
BEGIN
    DECLARE v_product_id BIGINT;
    DECLARE v_quantity INT;
    DECLARE v_price DECIMAL(12,2);
    DECLARE v_stock INT;
    DECLARE v_total DECIMAL(12,2) DEFAULT 0;
    DECLARE v_items INT;
    DECLARE v_i INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_order_id = -1;
```

```
        SET p_status = 'FAILED';
    END;

    SET v_items = JSON_LENGTH(p_product_ids);

    START TRANSACTION;

    -- Create order
    INSERT INTO orders (customer_id, total_amount, status, shipping_address,
created_at)
    VALUES (p_customer_id, 0, 'PENDING', p_shipping_address, NOW());
    SET p_order_id = LAST_INSERT_ID();

    -- Process each item
    WHILE v_i < v_items DO
        SET v_product_id = JSON_EXTRACT(p_product_ids, CONCAT('$[', v_i, ']'));
        SET v_quantity = JSON_EXTRACT(p_quantities, CONCAT('$[', v_i, ']'));

        -- Lock product row and check stock
        SELECT price, stock_quantity INTO v_price, v_stock
        FROM products WHERE id = v_product_id FOR UPDATE;

        IF v_stock < v_quantity THEN
            SET p_status = CONCAT('Insufficient stock for product ',
v_product_id);
            ROLLBACK;
            SET p_order_id = -1;
            LEAVE;
        END IF;

        -- Insert order item
        INSERT INTO order_items (order_id, product_id, quantity, unit_price)
        VALUES (p_order_id, v_product_id, v_quantity, v_price);

        -- Update stock
        UPDATE products SET stock_quantity = stock_quantity - v_quantity
        WHERE id = v_product_id;

        SET v_total = v_total + (v_price * v_quantity);
        SET v_i = v_i + 1;
    END WHILE;

    -- Update total
    UPDATE orders SET total_amount = v_total WHERE id = p_order_id;

    COMMIT;
    SET p_status = 'SUCCESS';
```

```
END //
DELIMITER ;
```

11.2 Banking Transaction System

```
-- Daily interest calculation
DELIMITER //
CREATE PROCEDURE CalculateDailyInterest()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_account_id BIGINT;
    DECLARE v_balance DECIMAL(15,2);
    DECLARE v_rate DECIMAL(8,6);
    DECLARE v_interest DECIMAL(15,2);
    DECLARE v_processed INT DEFAULT 0;

    DECLARE account_cursor CURSOR FOR
        SELECT a.id, a.balance, at.annual_interest_rate
        FROM accounts a
        JOIN account_types at ON a.type_id = at.id
        WHERE a.status = 'ACTIVE' AND a.balance > 0;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    START TRANSACTION;

    OPEN account_cursor;
    calc_loop: LOOP
        FETCH account_cursor INTO v_account_id, v_balance, v_rate;
        IF done THEN LEAVE calc_loop; END IF;

        -- Daily interest = balance * (annual_rate / 365)
        SET v_interest = ROUND(v_balance * (v_rate / 365), 2);

        -- Credit interest
        UPDATE accounts SET balance = balance + v_interest WHERE id =
v_account_id;

        -- Log transaction
        INSERT INTO transactions (account_id, amount, type, description,
created_at)
        VALUES (v_account_id, v_interest, 'INTEREST', 'Daily interest credit',
NOW());
```

```
        SET v_processed = v_processed + 1;
    END LOOP;
    CLOSE account_cursor;

    -- Log batch completion
    INSERT INTO batch_log (job_name, records_processed, completed_at)
    VALUES ('DAILY_INTEREST', v_processed, NOW());

    COMMIT;
END //
DELIMITER ;
```

11.3 Inventory Management Queries

```
-- Products running low on stock
SELECT p.id, p.name, p.sku, p.stock_quantity, p.category,
       CASE
           WHEN p.stock_quantity = 0 THEN 'OUT_OF_STOCK'
           WHEN p.stock_quantity < 10 THEN 'CRITICAL'
           WHEN p.stock_quantity < 50 THEN 'LOW'
           ELSE 'ADEQUATE'
       END AS stock_status
FROM products p
WHERE p.is_active = TRUE AND p.stock_quantity < 50
ORDER BY p.stock_quantity ASC;

-- Top selling products this month
SELECT p.name, p.sku,
       SUM(oi.quantity) AS units_sold,
       SUM(oi.line_total) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.id
JOIN orders o ON oi.order_id = o.id
WHERE o.status IN ('CONFIRMED', 'SHIPPED', 'DELIVERED')
  AND o.created_at >= DATE_FORMAT(NOW(), '%Y-%m-01')
GROUP BY p.id, p.name, p.sku
ORDER BY units_sold DESC
LIMIT 20;

-- Inventory turnover rate
SELECT p.name, p.stock_quantity,
       COALESCE(sales.monthly_sold, 0) AS monthly_sold,
       CASE
           WHEN sales.monthly_sold > 0
```

```sql
            THEN ROUND(p.stock_quantity / sales.monthly_sold, 1)
            ELSE 999
        END AS months_of_stock
FROM products p
LEFT JOIN (
    SELECT oi.product_id, SUM(oi.quantity) AS monthly_sold
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.id
    WHERE o.created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
    GROUP BY oi.product_id
) sales ON p.id = sales.product_id
WHERE p.is_active = TRUE
ORDER BY months_of_stock ASC;
```

11.4 Batch Processing: Monthly Report

```sql
-- Generate monthly revenue report
CREATE TABLE monthly_reports (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    report_month DATE NOT NULL,
    total_orders INT,
    total_revenue DECIMAL(15,2),
    avg_order_value DECIMAL(12,2),
    new_customers INT,
    top_category VARCHAR(100),
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE PROCEDURE GenerateMonthlyReport(
    IN p_year INT,
    IN p_month INT
)
BEGIN
    DECLARE v_start_date DATE;
    DECLARE v_end_date DATE;
    DECLARE v_total_orders INT;
    DECLARE v_total_revenue DECIMAL(15,2);
    DECLARE v_avg_value DECIMAL(12,2);
    DECLARE v_new_customers INT;
    DECLARE v_top_category VARCHAR(100);

    SET v_start_date = CONCAT(p_year, '-', LPAD(p_month, 2, '0'), '-01');
    SET v_end_date = LAST_DAY(v_start_date);
```

```
    -- Order stats
    SELECT COUNT(*), COALESCE(SUM(total_amount), 0),
COALESCE(AVG(total_amount), 0)
    INTO v_total_orders, v_total_revenue, v_avg_value
    FROM orders
    WHERE created_at BETWEEN v_start_date AND v_end_date
      AND status IN ('CONFIRMED', 'SHIPPED', 'DELIVERED');

    -- New customers
    SELECT COUNT(*) INTO v_new_customers
    FROM customers
    WHERE created_at BETWEEN v_start_date AND v_end_date;

    -- Top category by revenue
    SELECT p.category INTO v_top_category
    FROM order_items oi
    JOIN products p ON oi.product_id = p.id
    JOIN orders o ON oi.order_id = o.id
    WHERE o.created_at BETWEEN v_start_date AND v_end_date
    GROUP BY p.category
    ORDER BY SUM(oi.line_total) DESC
    LIMIT 1;

    INSERT INTO monthly_reports
        (report_month, total_orders, total_revenue, avg_order_value,
new_customers, top_category)
    VALUES
        (v_start_date, v_total_orders, v_total_revenue, v_avg_value,
v_new_customers, v_top_category);
END //
DELIMITER ;
```

Part 12: MySQL + Spring Boot Integration

12.1 JDBC (Java Database Connectivity)

```
// Raw JDBC -- low-level, manual connection management
public class JdbcExample {

    public User findById(Long id) throws SQLException {
        String sql = "SELECT id, name, email FROM users WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setLong(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return new User(
                        rs.getLong("id"),
                        rs.getString("name"),
                        rs.getString("email")
                    );
                }
            }
        }
        return null;
    }
}
// Problems: boilerplate, manual resource management, no caching
```

12.2 Spring JdbcTemplate

```
@Repository
public class UserJdbcRepository {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    public User findById(Long id) {
        String sql = "SELECT id, name, email, salary FROM users WHERE id = ?";
        return jdbcTemplate.queryForObject(sql, (rs, rowNum) -> new User(
```

```
            rs.getLong("id"),
            rs.getString("name"),
            rs.getString("email"),
            rs.getBigDecimal("salary")
        ), id);
    }

    public List<User> findByDepartment(String department) {
        String sql = "SELECT * FROM users WHERE department = ? ORDER BY name";
        return jdbcTemplate.query(sql, (rs, rowNum) -> new User(
            rs.getLong("id"),
            rs.getString("name"),
            rs.getString("email"),
            rs.getBigDecimal("salary")
        ), department);
    }

    public int updateSalary(Long id, BigDecimal newSalary) {
        return jdbcTemplate.update(
            "UPDATE users SET salary = ? WHERE id = ?", newSalary, id);
    }
}
```

12.3 Connection Pooling (HikariCP)

```yaml
# application.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb?useSSL=true&serverTimezone=UTC
    username: app_user
    password: secret
    driver-class-name: com.mysql.cj.jdbc.Driver
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      idle-timeout: 300000        # 5 min
      connection-timeout: 20000   # 20 sec
      max-lifetime: 1800000       # 30 min
      leak-detection-threshold: 60000  # Detect connection leaks
      pool-name: MyAppPool
```

```
Connection Pool Flow:

Application needs DB connection
        |
        v
+------------------+
| HikariCP Pool    |
| (20 connections) |
+------------------+
  |  Borrow        |  Return
  v                v
+------------------+
| MySQL Server     |
| (max_connections)|
+------------------+

Benefits:
- Reuse connections (creating new ones is expensive: ~100ms)
- Limit total connections to prevent DB overload
- Detect leaked connections
- Auto-validate connections before use
```

12.4 Hibernate + Spring Data JPA

```java
// Entity
@Entity
@Table(name = "orders",
       indexes = @Index(name = "idx_customer_status", columnList =
"customer_id, status"))
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "customer_id", nullable = false)
    private Long customerId;

    @Column(precision = 12, scale = 2)
    private BigDecimal totalAmount;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    @Column(updatable = false)
```

```
    private LocalDateTime createdAt;

    @PrePersist
    void onCreate() { createdAt = LocalDateTime.now(); }
}

// Repository
public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByCustomerIdOrderByCreatedAtDesc(Long customerId);

    @Query("SELECT o FROM Order o WHERE o.status = :status " +
           "AND o.createdAt > :since")
    Page<Order> findRecentByStatus(@Param("status") OrderStatus status,
                                   @Param("since") LocalDateTime since,
                                   Pageable pageable);

    @Modifying @Transactional
    @Query("UPDATE Order o SET o.status = 'CANCELLED' " +
           "WHERE o.status = 'PENDING' AND o.createdAt < :cutoff")
    int cancelStaleOrders(@Param("cutoff") LocalDateTime cutoff);
}
```

12.5 Calling Stored Procedures from Spring Boot

```
@Repository
public class PaymentRepository {

    @PersistenceContext
    private EntityManager em;

    public TransferResult transferFunds(Long fromId, Long toId, BigDecimal
amount) {
        StoredProcedureQuery query = em
            .createStoredProcedureQuery("TransferFunds")
            .registerStoredProcedureParameter("from_account_id", Long.class,
ParameterMode.IN)
            .registerStoredProcedureParameter("to_account_id", Long.class,
ParameterMode.IN)
            .registerStoredProcedureParameter("amount", BigDecimal.class,
ParameterMode.IN)
            .registerStoredProcedureParameter("transfer_id", Long.class,
ParameterMode.OUT)
            .registerStoredProcedureParameter("status_message", String.class,
ParameterMode.OUT)
```

```
            .setParameter("from_account_id", fromId)
            .setParameter("to_account_id", toId)
            .setParameter("amount", amount);

        query.execute();

        Long transferId = (Long) query.getOutputParameterValue("transfer_id");
        String status = (String)
query.getOutputParameterValue("status_message");

        return new TransferResult(transferId, status);
    }
}
```

Part 13: Stored Procedure Production Examples

13.1 Payment Processing

```
DELIMITER //
CREATE PROCEDURE ProcessPayment(
    IN p_order_id BIGINT,
    IN p_payment_method VARCHAR(20),
    IN p_card_last_four VARCHAR(4),
    OUT p_payment_id BIGINT,
    OUT p_result VARCHAR(50)
)
BEGIN
    DECLARE v_order_amount DECIMAL(12,2);
    DECLARE v_order_status VARCHAR(20);
    DECLARE v_customer_id BIGINT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_result = 'PAYMENT_FAILED';
        SET p_payment_id = -1;
    END;

    START TRANSACTION;

    -- Get order details with exclusive lock
    SELECT total_amount, status, customer_id
    INTO v_order_amount, v_order_status, v_customer_id
    FROM orders WHERE id = p_order_id FOR UPDATE;

    -- Validate order
    IF v_order_status != 'PENDING' THEN
        SET p_result = 'ORDER_NOT_PENDING';
        SET p_payment_id = -1;
        ROLLBACK;
    ELSE
        -- Create payment record
        INSERT INTO payments (order_id, customer_id, amount, payment_method,
                              card_last_four, status, created_at)
        VALUES (p_order_id, v_customer_id, v_order_amount, p_payment_method,
                p_card_last_four, 'SUCCESS', NOW());
```

```
        SET p_payment_id = LAST_INSERT_ID();

        -- Update order status
        UPDATE orders SET status = 'CONFIRMED', updated_at = NOW()
        WHERE id = p_order_id;

        -- Create transaction log
        INSERT INTO transaction_log (payment_id, action, amount, created_at)
        VALUES (p_payment_id, 'CHARGE', v_order_amount, NOW());

        SET p_result = 'SUCCESS';
        COMMIT;
    END IF;
END //
DELIMITER ;
```

13.2 Eligibility Calculation (Government System)

```
DELIMITER //
CREATE PROCEDURE CheckEligibility(
    IN p_applicant_id BIGINT,
    OUT p_eligible BOOLEAN,
    OUT p_reason VARCHAR(200),
    OUT p_benefit_amount DECIMAL(10,2)
)
BEGIN
    DECLARE v_age INT;
    DECLARE v_income DECIMAL(12,2);
    DECLARE v_household_size INT;
    DECLARE v_state VARCHAR(2);
    DECLARE v_existing_benefits INT;
    DECLARE v_fpl DECIMAL(12,2);

    -- Get applicant info
    SELECT
        TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()),
        annual_income,
        household_size,
        state_code
    INTO v_age, v_income, v_household_size, v_state
    FROM applicants
    WHERE id = p_applicant_id;

    -- Check for existing active benefits
```

```
    SELECT COUNT(*) INTO v_existing_benefits
    FROM benefits
    WHERE applicant_id = p_applicant_id AND status = 'ACTIVE';

    -- Get Federal Poverty Level for household size
    SELECT income_threshold INTO v_fpl
    FROM poverty_levels
    WHERE household_size = LEAST(v_household_size, 8)
      AND effective_year = YEAR(CURDATE());

    -- Eligibility rules
    SET p_eligible = FALSE;
    SET p_benefit_amount = 0;

    IF v_existing_benefits > 0 THEN
        SET p_reason = 'Already receiving benefits';
    ELSEIF v_age < 18 THEN
        SET p_reason = 'Applicant must be 18 or older';
    ELSEIF v_income > (v_fpl * 1.38) THEN
        SET p_reason = 'Income exceeds 138% of Federal Poverty Level';
    ELSE
        SET p_eligible = TRUE;
        SET p_reason = 'Eligible';

        -- Calculate benefit amount based on income bracket
        IF v_income <= (v_fpl * 0.5) THEN
            SET p_benefit_amount = 500.00 * v_household_size;
        ELSEIF v_income <= v_fpl THEN
            SET p_benefit_amount = 350.00 * v_household_size;
        ELSE
            SET p_benefit_amount = 200.00 * v_household_size;
        END IF;

        -- Log eligibility determination
        INSERT INTO eligibility_log
            (applicant_id, eligible, reason, benefit_amount, determined_at)
        VALUES
            (p_applicant_id, p_eligible, p_reason, p_benefit_amount, NOW());
    END IF;
END //
DELIMITER ;
```

13.3 Report Generation

```sql
DELIMITER //
CREATE PROCEDURE GenerateSalesReport(
    IN p_start_date DATE,
    IN p_end_date DATE
)
BEGIN
    -- Summary
    SELECT
        COUNT(DISTINCT o.id) AS total_orders,
        COUNT(DISTINCT o.customer_id) AS unique_customers,
        SUM(o.total_amount) AS total_revenue,
        AVG(o.total_amount) AS avg_order_value,
        MAX(o.total_amount) AS largest_order
    FROM orders o
    WHERE o.created_at BETWEEN p_start_date AND p_end_date
      AND o.status IN ('CONFIRMED', 'SHIPPED', 'DELIVERED');

    -- Daily breakdown
    SELECT
        DATE(o.created_at) AS order_date,
        COUNT(*) AS orders,
        SUM(o.total_amount) AS revenue
    FROM orders o
    WHERE o.created_at BETWEEN p_start_date AND p_end_date
      AND o.status IN ('CONFIRMED', 'SHIPPED', 'DELIVERED')
    GROUP BY DATE(o.created_at)
    ORDER BY order_date;

    -- Top products
    SELECT
        p.name,
        p.category,
        SUM(oi.quantity) AS units_sold,
        SUM(oi.line_total) AS revenue,
        RANK() OVER (ORDER BY SUM(oi.line_total) DESC) AS revenue_rank
    FROM order_items oi
    JOIN products p ON oi.product_id = p.id
    JOIN orders o ON oi.order_id = o.id
    WHERE o.created_at BETWEEN p_start_date AND p_end_date
    GROUP BY p.id, p.name, p.category
    ORDER BY revenue DESC
    LIMIT 20;

    -- Category performance
    SELECT
        p.category,
```

```
        COUNT(DISTINCT o.id) AS orders,
        SUM(oi.quantity) AS units,
        SUM(oi.line_total) AS revenue,
        ROUND(SUM(oi.line_total) / (SELECT SUM(total_amount) FROM orders
            WHERE created_at BETWEEN p_start_date AND p_end_date) * 100, 2) AS
pct_of_total
    FROM order_items oi
    JOIN products p ON oi.product_id = p.id
    JOIN orders o ON oi.order_id = o.id
    WHERE o.created_at BETWEEN p_start_date AND p_end_date
    GROUP BY p.category
    ORDER BY revenue DESC;
END //
DELIMITER ;
```

13.4 Batch Data Processing

```
DELIMITER //
CREATE PROCEDURE BatchDeactivateInactiveUsers(
    IN p_inactive_days INT,
    OUT p_count INT
)
BEGIN
    DECLARE v_batch_size INT DEFAULT 1000;
    DECLARE v_affected INT DEFAULT 1;

    SET p_count = 0;

    -- Process in batches to avoid long-running locks
    WHILE v_affected > 0 DO
        UPDATE users
        SET status = 'INACTIVE', updated_at = NOW()
        WHERE status = 'ACTIVE'
          AND last_login_at < DATE_SUB(NOW(), INTERVAL p_inactive_days DAY)
        LIMIT v_batch_size;

        SET v_affected = ROW_COUNT();
        SET p_count = p_count + v_affected;

        -- Small delay to let other transactions proceed
        IF v_affected > 0 THEN
            DO SLEEP(0.1);
        END IF;
    END WHILE;
```

```
END //
DELIMITER ;
```

Part 14: Database Performance Tuning

14.1 Query Tuning Checklist

Step	Action
1	Run EXPLAIN on slow queries
2	Check if indexes exist for WHERE/JOIN columns
3	Look for full table scans (type: ALL)
4	Check for "Using filesort" and "Using temporary"
5	Verify selectivity of existing indexes
6	Consider covering indexes
7	Rewrite correlated subqueries as JOINs
8	Use LIMIT for large result sets
9	Avoid SELECT *
10	Check slow query log

14.2 Index Tuning

```sql
-- Find unused indexes
SELECT
    s.TABLE_NAME, s.INDEX_NAME,
    s.SEQ_IN_INDEX, s.COLUMN_NAME,
    t.TABLE_ROWS
FROM information_schema.STATISTICS s
JOIN information_schema.TABLES t
    ON s.TABLE_SCHEMA = t.TABLE_SCHEMA AND s.TABLE_NAME = t.TABLE_NAME
WHERE s.TABLE_SCHEMA = 'mydb'
ORDER BY t.TABLE_ROWS DESC;

-- Find missing indexes (queries with full table scans)
-- Enable slow query log:
SET GLOBAL slow_query_log = 'ON';
```

```sql
SET GLOBAL long_query_time = 1;  -- Log queries taking > 1 second
SET GLOBAL log_queries_not_using_indexes = 'ON';
```

14.3 Database Partitioning

```sql
-- Range partitioning by date (for large tables like orders)
CREATE TABLE orders_partitioned (
    id BIGINT AUTO_INCREMENT,
    customer_id BIGINT NOT NULL,
    total_amount DECIMAL(12,2),
    status VARCHAR(20),
    created_at DATETIME NOT NULL,
    PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (YEAR(created_at)) (
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p2023 VALUES LESS THAN (2024),
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION p_future VALUES LESS THAN MAXVALUE
);

-- Benefits:
-- Query for 2024 data only scans partition p2024
-- Old data can be dropped by dropping a single partition
-- Each partition has its own indexes
```

14.4 Read Replicas

```
Write traffic:
  Application -> Primary MySQL (writes)

Read traffic:
  Application -> Read Replica 1 (reads)
              -> Read Replica 2 (reads)

Primary MySQL --> Replication --> Replica 1
              --> Replication --> Replica 2

Spring Boot config:
```

```
@Transactional(readOnly = true)  -> Routes to read replica
@Transactional                    -> Routes to primary
```

```java
// Routing DataSource
public class ReadWriteRoutingDataSource extends AbstractRoutingDataSource {
    @Override
    protected Object determineCurrentLookupKey() {
        return TransactionSynchronizationManager.isCurrentTransactionReadOnly()
            ? "read" : "write";
    }
}
```

14.5 Server Configuration Tuning

```conf
# my.cnf (MySQL configuration)
[mysqld]
# InnoDB Buffer Pool (allocate 70-80% of server RAM)
innodb_buffer_pool_size = 8G

# Log file size (for crash recovery)
innodb_log_file_size = 512M

# Max connections
max_connections = 200

# Thread cache
thread_cache_size = 50

# Query cache (disabled in MySQL 8.0+)
# query_cache_type = 0

# Slow query log
slow_query_log = 1
long_query_time = 2
log_queries_not_using_indexes = 1

# Temp table size
tmp_table_size = 256M
max_heap_table_size = 256M

# Sort buffer
```

```
sort_buffer_size = 4M
join_buffer_size = 4M
```

Part 15: Common Production Issues

15.1 Deadlocks

Symptoms: Application throws `com.mysql.cj.jdbc.exceptions.MySQLTransactionRollbackException: Deadlock found`

Detection:

```
SHOW ENGINE INNODB STATUS;  -- Shows last deadlock info
-- Look for "LATEST DETECTED DEADLOCK" section
```

Prevention:

```
-- 1. Always access rows in the same order
-- BAD:
-- T1: UPDATE accounts WHERE id = 1; UPDATE accounts WHERE id = 2;
-- T2: UPDATE accounts WHERE id = 2; UPDATE accounts WHERE id = 1;

-- GOOD:
-- Both transactions lock in id order:
-- T1: UPDATE accounts WHERE id = 1; UPDATE accounts WHERE id = 2;
-- T2: UPDATE accounts WHERE id = 1; UPDATE accounts WHERE id = 2;

-- 2. Keep transactions short
-- 3. Use SELECT ... FOR UPDATE only when necessary
-- 4. Add proper indexes to reduce lock scope
```

15.2 Lock Contention

Symptoms: High wait times, slow queries, connection pool exhaustion.

```
-- Find blocking transactions
SELECT
    r.trx_id AS waiting_trx_id,
    r.trx_mysql_thread_id AS waiting_thread,
```

```sql
    r.trx_query AS waiting_query,
    b.trx_id AS blocking_trx_id,
    b.trx_mysql_thread_id AS blocking_thread,
    b.trx_query AS blocking_query
FROM information_schema.INNODB_LOCK_WAITS w
INNER JOIN information_schema.INNODB_TRX b ON b.trx_id = w.blocking_trx_id
INNER JOIN information_schema.INNODB_TRX r ON r.trx_id = w.requesting_trx_id;

-- Kill blocking transaction if necessary
KILL <blocking_thread_id>;
```

15.3 Slow Queries

Detection:

```sql
-- Check currently running queries
SHOW PROCESSLIST;

-- From slow query log
SELECT * FROM mysql.slow_log
ORDER BY query_time DESC LIMIT 10;

-- Performance Schema (MySQL 8.0+)
SELECT
    DIGEST_TEXT,
    COUNT_STAR AS exec_count,
    ROUND(SUM_TIMER_WAIT / 1000000000000, 2) AS total_time_sec,
    ROUND(AVG_TIMER_WAIT / 1000000000000, 2) AS avg_time_sec,
    SUM_ROWS_EXAMINED AS rows_examined
FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC
LIMIT 10;
```

Common fixes:

1. Add indexes on filtered/joined columns
2. Rewrite subqueries as JOINs
3. Use pagination instead of loading all rows
4. Add covering indexes
5. Optimize GROUP BY with indexes

15.4 High CPU Usage

Causes: Full table scans, missing indexes, complex queries, too many concurrent connections.

```sql
-- Find resource-intensive queries
SELECT * FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC LIMIT 5;

-- Check table scan ratio
SHOW GLOBAL STATUS LIKE 'Handler_read%';
-- Handler_read_rnd_next should be low relative to Handler_read_key
```

15.5 Connection Leaks

Symptoms: "Too many connections" errors, pool exhaustion.

```sql
-- Check current connections
SHOW GLOBAL STATUS LIKE 'Threads_connected';
SHOW GLOBAL STATUS LIKE 'Max_used_connections';
SHOW PROCESSLIST;  -- See all active connections
```

Prevention:

```yaml
# HikariCP leak detection
spring:
  datasource:
    hikari:
      leak-detection-threshold: 30000  # Warn if connection held > 30 sec
```

```java
// Always use try-with-resources or @Transactional
// BAD: Connection never returned to pool
Connection conn = dataSource.getConnection();
// ... use conn but forget to close

// GOOD:
try (Connection conn = dataSource.getConnection()) {
    // Auto-closed when block exits
}
```

```
// BEST: Let Spring manage connections
@Transactional
public void processData() {
    // Spring handles connection lifecycle
}
```

15.6 Troubleshooting Checklist

Issue	Command	Look For
Slow queries	`EXPLAIN`	type: ALL, Using filesort
Deadlocks	`SHOW ENGINE INNODB STATUS`	LATEST DETECTED DEADLOCK
Locks	`SHOW PROCESSLIST`	State: Locked, Waiting
Connections	`SHOW GLOBAL STATUS LIKE 'Threads%'`	Max exceeded
Memory	`SHOW VARIABLES LIKE 'innodb_buffer_pool_size'`	Pool too small
Replication lag	`SHOW SLAVE STATUS`	Seconds_Behind_Master

Part 16: Build It Yourself -- E-Commerce Database

> ***Goal:*** *Design and build a complete e-commerce database from scratch -- schema, indexes, stored procedures, and Spring Boot integration.*

Concept Overview

```
Tables:  customers -> orders -> order_items -> products
                                categories <--+
```

Table	Purpose	Key Indexes
`customers`	Customer accounts	email (UNIQUE), status
`products`	Product catalog	sku (UNIQUE), category_id + price
`categories`	Product categories	name (UNIQUE)
`orders`	Customer orders	customer_id + created_at, status
`order_items`	Line items per order	order_id, product_id

Step 1: Create the Database Schema

Concept: Design tables with proper data types, constraints, and indexes from the start.

```sql
CREATE DATABASE ecommerce CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE ecommerce;

-- Customers table
CREATE TABLE customers (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,          -- UNIQUE = auto-creates index
    phone VARCHAR(20),
```

```sql
    status ENUM('ACTIVE','INACTIVE','BLOCKED') DEFAULT 'ACTIVE',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_customer_status (status)            -- Fast filtering by status
);

-- Categories table
CREATE TABLE categories (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    parent_id INT,
    FOREIGN KEY (parent_id) REFERENCES categories(id)
);

-- Products table
CREATE TABLE products (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    sku VARCHAR(50) NOT NULL UNIQUE,
    name VARCHAR(200) NOT NULL,
    category_id INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT DEFAULT 0,
    status ENUM('ACTIVE','DISCONTINUED') DEFAULT 'ACTIVE',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES categories(id),
    INDEX idx_product_category_price (category_id, price)  -- Composite index
);

-- Orders table
CREATE TABLE orders (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    customer_id BIGINT NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    status ENUM('PENDING','CONFIRMED','SHIPPED','DELIVERED','CANCELLED')
DEFAULT 'PENDING',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id),
    INDEX idx_order_customer_date (customer_id, created_at DESC)
);

-- Order items table
CREATE TABLE order_items (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    line_total DECIMAL(12,2) GENERATED ALWAYS AS (quantity * unit_price)
```

```
STORED,
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);
```

Step 2: Insert Sample Data

```
INSERT INTO categories (name) VALUES ('Electronics'), ('Clothing'), ('Books');

INSERT INTO products (sku, name, category_id, price, stock_quantity) VALUES
('ELEC-001', 'Wireless Mouse', 1, 29.99, 500),
('ELEC-002', 'USB-C Hub', 1, 49.99, 200),
('CLT-001', 'Cotton T-Shirt', 2, 19.99, 1000),
('BK-001', 'Java Concurrency in Practice', 3, 45.00, 150);

INSERT INTO customers (first_name, last_name, email) VALUES
('John', 'Doe', 'john@example.com'),
('Jane', 'Smith', 'jane@example.com');
```

Step 3: Write Essential Queries

```
-- Top-selling products by revenue
SELECT p.name, SUM(oi.quantity) AS units_sold, SUM(oi.line_total) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.id
GROUP BY p.id ORDER BY revenue DESC LIMIT 10;

-- Customer order history with totals
SELECT c.first_name, c.last_name, COUNT(o.id) AS order_count,
       SUM(o.total_amount) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
GROUP BY c.id ORDER BY total_spent DESC;

-- Low stock alert
SELECT sku, name, stock_quantity FROM products
WHERE stock_quantity < 10 AND status = 'ACTIVE';
```

Step 4: Create a Stored Procedure

```
DELIMITER //
CREATE PROCEDURE PlaceOrder(
    IN p_customer_id BIGINT,
    IN p_product_id BIGINT,
    IN p_quantity INT,
    OUT p_order_id BIGINT,
    OUT p_result VARCHAR(50)
)
BEGIN
    DECLARE v_price DECIMAL(10,2);
    DECLARE v_stock INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_result = 'ERROR';
    END;

    START TRANSACTION;

    SELECT price, stock_quantity INTO v_price, v_stock
    FROM products WHERE id = p_product_id FOR UPDATE;

    IF v_stock < p_quantity THEN
        SET p_result = 'INSUFFICIENT_STOCK';
        ROLLBACK;
    ELSE
        INSERT INTO orders (customer_id, total_amount, status)
        VALUES (p_customer_id, v_price * p_quantity, 'CONFIRMED');
        SET p_order_id = LAST_INSERT_ID();

        INSERT INTO order_items (order_id, product_id, quantity, unit_price)
        VALUES (p_order_id, p_product_id, p_quantity, v_price);

        UPDATE products SET stock_quantity = stock_quantity - p_quantity
        WHERE id = p_product_id;

        SET p_result = 'SUCCESS';
        COMMIT;
    END IF;
END //
DELIMITER ;
```

Step 5: Call from Spring Boot

```
@Repository
public class OrderProcedureRepository {
    @PersistenceContext private EntityManager em;

    public OrderResult placeOrder(Long customerId, Long productId, int qty) {
        StoredProcedureQuery query =
em.createStoredProcedureQuery("PlaceOrder")
            .registerStoredProcedureParameter("p_customer_id", Long.class,
ParameterMode.IN)
            .registerStoredProcedureParameter("p_product_id", Long.class,
ParameterMode.IN)
            .registerStoredProcedureParameter("p_quantity", Integer.class,
ParameterMode.IN)
            .registerStoredProcedureParameter("p_order_id", Long.class,
ParameterMode.OUT)
            .registerStoredProcedureParameter("p_result", String.class,
ParameterMode.OUT)
            .setParameter("p_customer_id", customerId)
            .setParameter("p_product_id", productId)
            .setParameter("p_quantity", qty);
        query.execute();
        return new OrderResult(
            (Long) query.getOutputParameterValue("p_order_id"),
            (String) query.getOutputParameterValue("p_result"));
    }
}
```

Key Takeaways

Concept	Remember
Index design	Create indexes on columns used in WHERE, JOIN, ORDER BY
Composite indexes	Order matters: `(customer_id, created_at)` ≠ `(created_at, customer_id)`
`FOR UPDATE`	Locks rows during transaction -- prevents concurrent modification
Stored procedures	Atomic operations that run entirely in the database

`EXPLAIN`	Always check query plans before production deployment

Part 16: SQL Interview Questions (100+)

SQL Basics (Q1-Q20)

Q1. What is the difference between DDL and DML? DDL (Data Definition Language) defines structure: CREATE, ALTER, DROP, TRUNCATE. DML (Data Manipulation Language) manipulates data: SELECT, INSERT, UPDATE, DELETE. DDL auto-commits; DML can be rolled back within a transaction.

Q2. What is the difference between DELETE, TRUNCATE, and DROP? DELETE: removes rows (can use WHERE, logged, can rollback). TRUNCATE: removes all rows (no WHERE, faster, resets AUTO_INCREMENT). DROP: removes entire table (structure + data). DELETE is slowest; TRUNCATE is fastest for clearing a table.

Q3. What is the difference between WHERE and HAVING? WHERE filters rows before GROUP BY. HAVING filters groups after GROUP BY. WHERE cannot use aggregate functions; HAVING can. Example: `HAVING COUNT(*) > 5`.

Q4. What is the difference between CHAR and VARCHAR? CHAR: fixed-length (always allocates full size, padded with spaces). VARCHAR: variable-length (stores only actual characters + 1-2 bytes for length). Use CHAR for fixed data (state codes, zip codes); VARCHAR for variable data (names, emails).

Q5. What is the difference between UNION and UNION ALL? UNION combines results and removes duplicates (slower, requires sort). UNION ALL combines results and keeps all duplicates (faster). Use UNION ALL when you know there are no duplicates or duplicates are acceptable.

Q6. What are constraints? Rules enforced on columns: PRIMARY KEY (unique + not null), FOREIGN KEY (referential integrity), UNIQUE (no duplicates), NOT NULL (required), CHECK (condition), DEFAULT (fallback value).

Q7. What is a primary key? A column (or combination) that uniquely identifies each row. Must be unique and NOT NULL. Only one primary key per table. InnoDB stores data in primary key order (clustered index).

Q8. What is a foreign key? A column that references the primary key of another table. Enforces referential integrity. Prevents orphan records. Can specify ON DELETE CASCADE or ON DELETE SET NULL.

Q9. What is AUTO_INCREMENT? MySQL feature that automatically generates a unique integer for new rows. Used for primary keys. Starts at 1 by default, increments by 1. Thread-safe.

Q10. What is the difference between IN and EXISTS? IN compares a value against a list. EXISTS checks if a subquery returns any rows. EXISTS is usually faster for large subquery results because it stops at the first match.

Q11. Write a query to find the second highest salary.

```sql
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
-- Or using DENSE_RANK:
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked WHERE rnk = 2;
```

Q12. What is the difference between JOIN and subquery? JOINs are generally faster because the optimizer handles them better. Subqueries are more readable for simple lookups. Correlated subqueries are slow (execute once per row). Prefer JOINs for large datasets.

Q13. What is NULL in SQL? NULL represents unknown or missing data. NULL is not zero, not empty string. Any comparison with NULL returns NULL (not TRUE). Use IS NULL / IS NOT NULL to check. Use COALESCE to handle NULLs.

Q14. What is COALESCE? Returns the first non-NULL value from a list of expressions.

```sql
SELECT COALESCE(phone, mobile, 'No phone') AS contact FROM customers;
```

Q15. What is CASE expression? SQL's if-then-else logic.

```sql
SELECT name,
    CASE
        WHEN salary > 100000 THEN 'Senior'
        WHEN salary > 60000 THEN 'Mid'
        ELSE 'Junior'
    END AS level
FROM employees;
```

Q16. What are aggregate functions? Functions that compute a single result from multiple rows: COUNT, SUM, AVG, MIN, MAX. Used with GROUP BY. Ignore NULLs (except COUNT(*)).

Q17. What is the execution order of a SQL query? FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT. This is why you can't use column aliases in WHERE but can in ORDER BY.

Q18. What is a view? A virtual table defined by a SELECT query. Does not store data physically. Used for abstraction, security (expose limited columns), and simplifying complex queries.

Q19. What is the difference between a view and a table? Table stores data physically; view stores a query definition. Table takes disk space; view takes none. Table data persists; view data is computed on access.

Q20. What are TCL commands? Transaction Control Language: COMMIT (save changes), ROLLBACK (undo changes), SAVEPOINT (partial rollback point), SET TRANSACTION (configure isolation level).

Joins (Q21-Q35)

Q21. Explain all types of JOINs. INNER JOIN: only matching rows from both tables. LEFT JOIN: all left rows + matching right (NULL if no match). RIGHT JOIN: all right rows + matching left. FULL OUTER JOIN: all rows from both (MySQL uses UNION). CROSS JOIN: cartesian product.

Q22. What is a self join? Joining a table with itself. Common use: employee-manager hierarchy.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e LEFT JOIN employees m ON e.manager_id = m.id;
```

Q23. What is a cross join? Produces cartesian product: every row from table A combined with every row from table B. If A has 100 rows and B has 50, result has 5000 rows. Rarely used intentionally.

Q24. What is the difference between LEFT JOIN and RIGHT JOIN? LEFT JOIN keeps all rows from the left table. RIGHT JOIN keeps all rows from the right table. They are mirrors of each other. Convention: use LEFT JOIN consistently.

Q25. How to find rows in table A that don't exist in table B?

```
SELECT a.* FROM table_a a
LEFT JOIN table_b b ON a.id = b.a_id
WHERE b.id IS NULL;
-- Or: NOT EXISTS
SELECT * FROM table_a a
WHERE NOT EXISTS (SELECT 1 FROM table_b b WHERE b.a_id = a.id);
```

Q26. What is a natural join? Joins on columns with the same name in both tables automatically. Risky because schema changes can break it. Always use explicit JOIN ON conditions.

Q27. Can you JOIN more than 2 tables? Yes. JOINs chain: `FROM a JOIN b ON ... JOIN c ON ... JOIN d ON ...`. Performance depends on indexes and data volume. Each additional JOIN adds complexity.

Q28. What is the performance impact of JOINs? Depends on indexes, data size, and join type. Nested loop join is O(n*m) without indexes. With indexes, each lookup is O(log n). Always index JOIN columns.

Q29. How to optimize a slow JOIN?

1. Index columns in ON clause. 2. Filter early with WHERE. 3. Reduce selected columns. 4. Consider denormalization for read-heavy queries. 5. Use EXPLAIN to check plan.

Q30. What is the difference between ON and WHERE in a LEFT JOIN? ON condition determines which rows to join. WHERE filters after the join. Putting a condition on the RIGHT table in WHERE effectively turns LEFT JOIN into INNER JOIN.

Q31. Write a query to find duplicate rows.

```
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

Q32. Write a query to delete duplicate rows keeping one.

```
DELETE u1 FROM users u1
INNER JOIN users u2 ON u1.email = u2.email
WHERE u1.id > u2.id;
```

Q33. What is a semi-join? A join where you check existence but don't include columns from the second table. Implemented using EXISTS or IN. Optimizer may convert IN to semi-join.

Q34. What is an anti-join? A join that returns rows from the first table that have NO match in the second table. Implemented using NOT EXISTS or LEFT JOIN + IS NULL.

Q35. How does MySQL process multi-table JOINs? MySQL uses nested-loop join by default. For each row in the outer table, it scans/looks up matching rows in the inner table using indexes. The optimizer chooses table order for best performance.

Indexes (Q36-Q50)

Q36. What is an index? A data structure (B+ tree) that speeds up data retrieval by allowing direct look up instead of full table scan. Like a book's index that maps topics to page numbers.

Q37. What is the difference between clustered and non-clustered indexes? Clustered: data rows stored in index order (one per table, usually PK). Non-clustered: separate structure pointing to PK (multiple per table).

Q38. What is a composite index? An index on multiple columns. Follows leftmost prefix rule: `INDEX(a,b,c)` supports queries on (a), (a,b), (a,b,c) but NOT (b), (c), or (b,c).

Q39. What is a covering index? An index that contains all columns needed by a query. MySQL reads data entirely from the index without accessing the table. EXPLAIN shows "Using index".

Q40. When should you NOT create an index? Small tables (< 1000 rows), frequently updated columns, low cardinality columns (boolean), wide columns (TEXT/BLOB), tables with heavy writes and few reads.

Q41. What is index selectivity? The ratio of distinct values to total rows. High selectivity (close to 1.0) means the index is effective. Low selectivity (e.g., gender column) means the index is nearly useless.

Q42. How does a B+ tree index work? A balanced tree where leaf nodes contain pointers to data rows. Each lookup traverses from root to leaf (O(log n)). Leaf nodes are linked for efficient range scans.

Q43. What is a full-text index? A special index for text search. Supports natural language search and boolean mode. Used for searching large text columns.

```
CREATE FULLTEXT INDEX idx_content ON articles(title, body);
SELECT * FROM articles WHERE MATCH(title, body) AGAINST('database
optimization');
```

Q44. What is the impact of too many indexes? Slower writes (every INSERT/UPDATE/DELETE updates all indexes). More disk space. More memory usage. Only create indexes that are actually used by queries.

Q45. How to find unused indexes?

```
SELECT * FROM sys.schema_unused_indexes WHERE object_schema = 'mydb';
```

Q46. What is an index hint? Force or suggest MySQL to use a specific index.

```
SELECT * FROM orders USE INDEX (idx_customer_id) WHERE customer_id = 42;
SELECT * FROM orders FORCE INDEX (idx_date) WHERE created_at > '2024-01-01';
```

Q47. What is a prefix index? Indexing only the first N characters of a string column. Saves space. Useful for long strings.

```
CREATE INDEX idx_email ON users(email(50));
```

Q48. How to check if an index is being used? Use EXPLAIN. Check the "key" column. If NULL, no index is used. Also check "type": ALL means full scan.

Q49. What is an invisible index? MySQL 8.0 feature. Index exists but optimizer ignores it. Used to test impact of dropping an index without actually dropping it.

```
ALTER TABLE orders ALTER INDEX idx_status INVISIBLE;
```

Q50. What is the difference between UNIQUE index and PRIMARY KEY? Both enforce uniqueness. PRIMARY KEY: only one per table, implies NOT NULL, defines clustered index. UNIQUE: multiple per table, allows NULL (one NULL), creates non-clustered index.

Transactions and Stored Procedures (Q51-Q70)

Q51. What is ACID? Atomicity (all or nothing), Consistency (valid state), Isolation (concurrent transactions don't interfere), Durability (committed data survives crash).

Q52. Explain isolation levels. READ UNCOMMITTED (see uncommitted changes), READ COMMITTED (see only committed), REPEATABLE READ (consistent read within transaction, MySQL default), SERIALIZABLE (full isolation, slowest).

Q53. What is a deadlock? Two transactions waiting for each other's locks. MySQL detects and kills one. Prevent by: accessing rows in consistent order, keeping transactions short, using proper indexes.

Q54. What is a stored procedure? Precompiled SQL program stored in the database. Called with CALL. Supports parameters (IN/OUT/INOUT), variables, flow control, cursors, and error handling.

Q55. What is the difference between stored procedure and function? Procedure: called with CALL, can have OUT params, can return multiple result sets, can do DML. Function: called in SQL expressions, returns single value, no OUT params in standard SQL.

Q56. What is a cursor? A database object that traverses the result set row by row. Used in stored procedures for row-level processing. Slower than set-based operations.

Q57. What is a trigger? SQL code that automatically executes when INSERT/UPDATE/DELETE occurs. Used for audit logging, data validation, denormalization. Cannot be called manually.

Q58. What is the difference between BEFORE and AFTER triggers? BEFORE: fires before the operation, can modify NEW values. AFTER: fires after the operation, data is already saved.

Q59. Can a trigger call itself (recursive trigger)? MySQL does not allow recursive triggers by default. A trigger on table A cannot cause another trigger on table A to fire.

Q60. What is a stored function?

```
CREATE FUNCTION CalculateTax(amount DECIMAL(12,2)) RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    RETURN amount * 0.18;
END;
-- Usage: SELECT price, CalculateTax(price) AS tax FROM products;
```

Q61. How to handle errors in stored procedures?

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; END;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Custom error';
```

Q62. What is SAVEPOINT? A named point within a transaction. You can ROLLBACK TO SAVEPOINT without undoing the entire transaction. Useful for partial undo in complex procedures.

Q63. What is autocommit? MySQL's default mode where each SQL statement is a separate transaction that auto-commits. Disable with `SET autocommit = 0;` to manually control transactions.

Q64. What is two-phase commit? A protocol for distributed transactions: Phase 1 (Prepare) all participants vote YES/NO. Phase 2 (Commit) if all voted YES, commit; otherwise rollback all.

Q65. How to optimize stored procedures?

1. Use set-based operations instead of cursors. 2. Add indexes for queried columns. 3. Minimize variables. 4. Avoid dynamic SQL unless necessary. 5. Keep transactions short.

Q66. What is DETERMINISTIC in functions? A function is DETERMINISTIC if it always returns the same result for the same input. MySQL can cache results of deterministic functions. Mark functions correctly for optimization.

Q67. What is the difference between temporary table and CTE? Temporary table: physical table in tempdb, persists for session, can be indexed. CTE: virtual, exists only for single query, no indexing. Use temp tables for large intermediate results; CTEs for readability.

Q68. How to pass a list of IDs to a stored procedure? Use JSON or comma-separated string with FIND_IN_SET, or a temporary table.

```sql
-- JSON approach (MySQL 5.7+)
DELIMITER //
CREATE PROCEDURE GetUsersByIds(IN p_ids JSON)
BEGIN
    SELECT * FROM users u
    WHERE JSON_CONTAINS(p_ids, CAST(u.id AS JSON));
END //
```

Q69. What is SELECT FOR UPDATE? Acquires exclusive lock on selected rows. Other transactions wait until lock is released. Used to prevent concurrent modifications.

```sql
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;
-- Row is locked until COMMIT or ROLLBACK
```

Q70. How to debug a stored procedure? Use SELECT statements to output intermediate values (poor man's debugger). Use information_schema.routines to view procedure source. Log to a debug table.

Query Optimization (Q71-Q85)

Q71. What is EXPLAIN? Shows the query execution plan: which indexes are used, join order, estimated rows scanned, and cost. Essential for diagnosing slow queries.

Q72. What does "type: ALL" mean in EXPLAIN? Full table scan. Every row is read. Usually means a missing index. This is the worst join type.

Q73. What is "Using filesort" in EXPLAIN? MySQL couldn't use an index for sorting and must sort in memory or disk. Often means ORDER BY columns are not indexed. Can be slow for large result sets.

Q74. What is "Using temporary" in EXPLAIN? MySQL creates a temporary table to process the query. Common with DISTINCT, GROUP BY, or UNION. May indicate optimization opportunity.

Q75. How to optimize GROUP BY? Add an index that matches the GROUP BY columns. If `GROUP BY department`, add `INDEX(department)`. This allows MySQL to group using the index without sorting.

Q76. What is query caching? MySQL 5.x had query cache (deprecated in 8.0). Application-level caching with Redis is the modern alternative.

Q77. How to optimize LIKE queries? `LIKE 'John%'` can use index (prefix match). `LIKE '%John%'` cannot use index (full scan). For substring search, use FULLTEXT index.

Q78. What is index condition pushdown? InnoDB evaluates WHERE conditions at the storage engine level using the index, reducing the number of rows the server needs to examine. Enabled by default in MySQL 5.6+.

Q79. How to optimize COUNT queries? `COUNT(*)` counts all rows (uses smallest index). `COUNT(column)` counts non-NULL rows in that column. For approximate counts, use `SHOW TABLE STATUS`.

Q80. What is the difference between LIMIT 1 and EXISTS? Both stop after finding one row. EXISTS is semantically clearer for "does this exist?" checks. Performance is similar but EXISTS may be slightly faster.

Q81. How to optimize INSERT performance? Batch inserts: `INSERT INTO t VALUES (1,..), (2,..), (3,..)`. Disable indexes during bulk load. Use `LOAD DATA INFILE` for massive imports. Wrap in single transaction.

Q82. What is partition pruning? When querying a partitioned table, MySQL only scans relevant partitions. If `WHERE year = 2024` and table is partitioned by year, only the 2024 partition is scanned.

Q83. How to profile a query?

```
SET profiling = 1;
SELECT * FROM orders WHERE customer_id = 42;
SHOW PROFILE FOR QUERY 1;
-- Shows time spent in: parsing, optimizing, executing, sending
```

Q84. What is a derived table? A subquery in the FROM clause. MySQL materializes it as a temporary table. Can be replaced with CTE for readability.

```
SELECT dept, avg_sal FROM (
    SELECT department AS dept, AVG(salary) AS avg_sal
    FROM employees GROUP BY department
) AS dept_avgs WHERE avg_sal > 80000;
```

Q85. How to handle pagination for millions of rows? Keyset/seek pagination instead of OFFSET:

```
-- Instead of: LIMIT 10 OFFSET 999990 (scans 1M rows!)
-- Use: WHERE id > last_seen_id LIMIT 10 (index lookup, fast)
```

Architecture (Q86-Q105)

Q86. What is InnoDB buffer pool? In-memory cache for data and indexes. Larger buffer pool = more data cached = fewer disk reads. Set to 70-80% of server RAM. Most impactful MySQL setting.

Q87. What is the redo log? Write-ahead log for crash recovery. Changes written to redo log before data files. On crash, MySQL replays redo log to recover committed transactions. Ensures durability (D in ACID).

Q88. What is the undo log? Stores previous versions of modified rows. Used for ROLLBACK and MVCC. Allows other transactions to read old versions while a transaction modifies data.

Q89. What is the binary log (binlog)? Records all data modifications in order. Used for replication (replicas replay binlog events) and point-in-time recovery. Different from redo log (which is for crash recovery).

Q90. What is replication in MySQL? Primary-replica setup: primary handles writes, binlog events replicate to replicas. Replicas handle reads. Provides read scalability and high availability.

Q91. What is GTID replication? Global Transaction Identifier: each transaction has a unique ID across all servers. Simplifies failover because replicas know exactly which transactions they've applied.

Q92. What is the difference between row-based and statement-based replication? Row-based: replicates actual row changes (larger binlog, deterministic). Statement-based: replicates SQL statements (smaller, non-deterministic for some functions). Mixed: uses statement-based by default, switches to row-based when needed.

Q93. How to implement high availability in MySQL? MySQL Group Replication, InnoDB Cluster, or Galera Cluster for multi-master. ProxySQL or MySQL Router for connection routing. Automatic failover on primary failure.

Q94. What is sharding? Distributing data across multiple MySQL instances. Each shard holds a subset of data (e.g., users 1-1M on shard1, 1M-2M on shard2). Application routes queries to correct shard.

Q95. What is the difference between horizontal and vertical partitioning? Horizontal: split rows across partitions (by range, hash, list). Vertical: split columns into separate tables (normalize). Horizontal is for large tables; vertical is for wide tables.

Q96. How does MySQL handle concurrent writes? InnoDB uses row-level locking, MVCC, and isolation levels. Writers lock affected rows. Readers use MVCC snapshots. Deadlocks are detected and resolved automatically.

Q97. What is the purpose of the innodb_flush_log_at_trx_commit setting? Controls durability vs performance. Value 1: flush redo log to disk on every commit (safest, slowest). Value 2: flush to OS cache on commit (faster, less safe). Value 0: flush every second (fastest, risk of 1 second data loss).

Q98. What is online DDL? MySQL 5.6+ allows ALTER TABLE operations without blocking reads/writes. Uses algorithm INPLACE or INSTANT. Example: adding an index doesn't lock the table.

Q99. Design a database schema for a banking system. Tables: accounts, customers, transactions, account_types, branches. Use DECIMAL for monetary values. Foreign keys for integrity. Indexes on account_number, customer_id. Stored procedures for transfers.

Q100. When would you choose MySQL over PostgreSQL? MySQL: simpler replication, widely supported, faster for simple queries. PostgreSQL: better for complex queries (CTEs, window functions), JSON support, extensibility. Choose based on use case.

Q101. What is connection pooling and why is it important? Reusing database connections instead of creating new ones. Creating a connection takes ~100ms. Pool maintains ready connections. HikariCP is the fastest Java connection pool.

Q102. What is the slow query log? Logs queries that take longer than `long_query_time` seconds. Essential for production monitoring. Should always be enabled.

Q103. How to handle schema migrations in production? Use Flyway or Liquibase. Version all schema changes as scripts. Run migrations during deployment. Always use backward-compatible changes. Never use `ddl-auto: update`.

Q104. What is the difference between OLTP and OLAP? OLTP: Online Transaction Processing (fast reads/writes, normalized, row-oriented). OLAP: Online Analytical Processing (complex queries, denormalized, columnar storage). Most backend apps are OLTP.

Q105. What is MySQL 8.0's most important feature? Window functions, CTEs (WITH clause), JSON improvements, invisible indexes, instant DDL, default authentication change (caching_sha2_password), roles for access control.

Part 17: Hands-on SQL Exercises (30)

Beginner (Exercises 1-10)

Exercise 1: Create a database called `ecommerce` with tables: `customers`, `products`, `orders`, `order_items`.

```sql
CREATE DATABASE ecommerce;
USE ecommerce;
CREATE TABLE customers (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE products (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    price DECIMAL(12,2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    category VARCHAR(100)
);
CREATE TABLE orders (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    customer_id BIGINT NOT NULL,
    total_amount DECIMAL(12,2) DEFAULT 0,
    status VARCHAR(20) DEFAULT 'PENDING',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);
CREATE TABLE order_items (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);
```

Exercise 2: Insert 5 customers and 10 products.

```sql
INSERT INTO customers (name, email) VALUES
('Alice Johnson', 'alice@mail.com'),
('Bob Smith', 'bob@mail.com'),
('Carol White', 'carol@mail.com'),
('David Brown', 'david@mail.com'),
('Eve Davis', 'eve@mail.com');

INSERT INTO products (name, price, stock, category) VALUES
('Laptop', 999.99, 50, 'Electronics'),
('Phone', 699.99, 100, 'Electronics'),
('Headphones', 149.99, 200, 'Electronics'),
('Desk', 299.99, 30, 'Furniture'),
('Chair', 199.99, 45, 'Furniture'),
('Keyboard', 79.99, 150, 'Accessories'),
('Mouse', 29.99, 300, 'Accessories'),
('Monitor', 449.99, 40, 'Electronics'),
('Webcam', 89.99, 80, 'Electronics'),
('USB Hub', 24.99, 500, 'Accessories');
```

Exercise 3: Find all products in the Electronics category priced above $100.

```sql
SELECT * FROM products WHERE category = 'Electronics' AND price > 100 ORDER BY
price DESC;
```

Exercise 4: Count products per category.

```sql
SELECT category, COUNT(*) AS product_count, AVG(price) AS avg_price
FROM products GROUP BY category ORDER BY product_count DESC;
```

Exercise 5: Find customers who have never placed an order.

```sql
SELECT c.* FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.id IS NULL;
```

Exercise 6: Get the total revenue per month.

```sql
SELECT
    DATE_FORMAT(o.created_at, '%Y-%m') AS month,
    COUNT(*) AS total_orders,
    SUM(o.total_amount) AS revenue
FROM orders o
WHERE o.status != 'CANCELLED'
GROUP BY DATE_FORMAT(o.created_at, '%Y-%m')
ORDER BY month;
```

Exercise 7: Find the most expensive product in each category.

```sql
SELECT p.* FROM products p
INNER JOIN (
    SELECT category, MAX(price) AS max_price
    FROM products GROUP BY category
) max_p ON p.category = max_p.category AND p.price = max_p.max_price;
```

Exercise 8: List all orders with customer name and order total.

```sql
SELECT o.id AS order_id, c.name AS customer, o.total_amount, o.status,
o.created_at
FROM orders o
JOIN customers c ON o.customer_id = c.id
ORDER BY o.created_at DESC;
```

Exercise 9: Update stock after an order is placed.

```sql
UPDATE products SET stock = stock - 2 WHERE id = 1;
```

Exercise 10: Delete all cancelled orders older than 1 year.

```sql
DELETE FROM order_items
WHERE order_id IN (
    SELECT id FROM orders
    WHERE status = 'CANCELLED' AND created_at < DATE_SUB(NOW(), INTERVAL 1
YEAR)
);
```

```sql
DELETE FROM orders
WHERE status = 'CANCELLED' AND created_at < DATE_SUB(NOW(), INTERVAL 1 YEAR);
```

Intermediate (Exercises 11-20)

Exercise 11: Find the top 5 customers by total spending.

```sql
SELECT c.name, SUM(o.total_amount) AS total_spent, COUNT(o.id) AS order_count
FROM customers c
JOIN orders o ON c.id = o.customer_id
WHERE o.status NOT IN ('CANCELLED')
GROUP BY c.id, c.name
ORDER BY total_spent DESC
LIMIT 5;
```

Exercise 12: Find products that have never been ordered.

```sql
SELECT p.* FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.id
);
```

Exercise 13: Calculate order totals using SUM of line items.

```sql
SELECT oi.order_id,
       SUM(oi.quantity * oi.unit_price) AS calculated_total
FROM order_items oi
GROUP BY oi.order_id;
```

Exercise 14: Rank customers by total spending.

```sql
SELECT
    c.name,
    SUM(o.total_amount) AS total_spent,
    RANK() OVER (ORDER BY SUM(o.total_amount) DESC) AS spending_rank
FROM customers c
```

```
JOIN orders o ON c.id = o.customer_id
GROUP BY c.id, c.name;
```

Exercise 15: Find month-over-month revenue growth.

```
WITH monthly AS (
    SELECT
        DATE_FORMAT(created_at, '%Y-%m') AS month,
        SUM(total_amount) AS revenue
    FROM orders WHERE status != 'CANCELLED'
    GROUP BY DATE_FORMAT(created_at, '%Y-%m')
)
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) AS prev_month,
    ROUND((revenue - LAG(revenue) OVER (ORDER BY month))
        / LAG(revenue) OVER (ORDER BY month) * 100, 2) AS growth_pct
FROM monthly;
```

Exercise 16: Find the running total of orders.

```
SELECT
    id, customer_id, total_amount, created_at,
    SUM(total_amount) OVER (ORDER BY created_at) AS running_total
FROM orders
WHERE status != 'CANCELLED';
```

Exercise 17: Find customers who ordered in both January and February.

```
SELECT c.name FROM customers c
JOIN orders o ON c.id = o.customer_id
WHERE MONTH(o.created_at) IN (1, 2) AND YEAR(o.created_at) = 2024
GROUP BY c.id, c.name
HAVING COUNT(DISTINCT MONTH(o.created_at)) = 2;
```

Exercise 18: Create a product sales summary with category totals.

```sql
SELECT
    COALESCE(p.category, 'TOTAL') AS category,
    SUM(oi.quantity) AS units_sold,
    SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.id
JOIN orders o ON oi.order_id = o.id
WHERE o.status IN ('CONFIRMED', 'DELIVERED')
GROUP BY p.category WITH ROLLUP;
```

Exercise 19: Find products with stock below average.

```sql
SELECT * FROM products
WHERE stock < (SELECT AVG(stock) FROM products)
ORDER BY stock ASC;
```

Exercise 20: Pivot monthly sales by category.

```sql
SELECT
    DATE_FORMAT(o.created_at, '%Y-%m') AS month,
    SUM(CASE WHEN p.category = 'Electronics' THEN oi.quantity * oi.unit_price
ELSE 0 END) AS electronics,
    SUM(CASE WHEN p.category = 'Furniture' THEN oi.quantity * oi.unit_price
ELSE 0 END) AS furniture,
    SUM(CASE WHEN p.category = 'Accessories' THEN oi.quantity * oi.unit_price
ELSE 0 END) AS accessories
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
GROUP BY DATE_FORMAT(o.created_at, '%Y-%m')
ORDER BY month;
```

Advanced (Exercises 21-30)

Exercise 21: Write a stored procedure to process an order.

```sql
DELIMITER //
CREATE PROCEDURE PlaceSimpleOrder(
```

```
    IN p_customer_id BIGINT, IN p_product_id BIGINT,
    IN p_quantity INT, OUT p_order_id BIGINT)
BEGIN
    DECLARE v_price DECIMAL(12,2);
    DECLARE v_stock INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; SET p_order_id = -1;
END;

    START TRANSACTION;
    SELECT price, stock INTO v_price, v_stock FROM products WHERE id =
p_product_id FOR UPDATE;
    IF v_stock < p_quantity THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient stock';
    END IF;
    INSERT INTO orders (customer_id, total_amount, status) VALUES
(p_customer_id, v_price * p_quantity, 'CONFIRMED');
    SET p_order_id = LAST_INSERT_ID();
    INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(p_order_id, p_product_id, p_quantity, v_price);
    UPDATE products SET stock = stock - p_quantity WHERE id = p_product_id;
    COMMIT;
END //
DELIMITER ;
```

Exercise 22: Write a trigger for audit logging on orders.

```
DELIMITER //
CREATE TRIGGER trg_orders_audit
AFTER UPDATE ON orders FOR EACH ROW
BEGIN
    IF OLD.status != NEW.status THEN
        INSERT INTO audit_log (table_name, record_id, action, old_values,
new_values, changed_at)
        VALUES ('orders', NEW.id, 'STATUS_CHANGE',
                JSON_OBJECT('status', OLD.status),
                JSON_OBJECT('status', NEW.status), NOW());
    END IF;
END //
DELIMITER ;
```

Exercise 23: Write a recursive CTE to find employee hierarchy.

```sql
WITH RECURSIVE hierarchy AS (
    SELECT id, name, manager_id, 1 AS level, CAST(name AS CHAR(500)) AS path
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.id, e.name, e.manager_id, h.level + 1, CONCAT(h.path, ' > ',
e.name)
    FROM employees e JOIN hierarchy h ON e.manager_id = h.id
)
SELECT * FROM hierarchy ORDER BY path;
```

Exercise 24: Implement keyset pagination.

```sql
-- Page 1:
SELECT id, name, price FROM products ORDER BY id LIMIT 10;
-- Page N (last seen id = 10):
SELECT id, name, price FROM products WHERE id > 10 ORDER BY id LIMIT 10;
```

Exercise 25: Write a query using NTILE to bucket customers by spending.

```sql
SELECT name, total_spent,
    CASE quartile
        WHEN 1 THEN 'Bronze' WHEN 2 THEN 'Silver'
        WHEN 3 THEN 'Gold' WHEN 4 THEN 'Platinum'
    END AS tier
FROM (
    SELECT c.name, SUM(o.total_amount) AS total_spent,
           NTILE(4) OVER (ORDER BY SUM(o.total_amount)) AS quartile
    FROM customers c JOIN orders o ON c.id = o.customer_id
    GROUP BY c.id, c.name
) ranked;
```

Exercise 26: Find gaps in sequential IDs.

```sql
SELECT t1.id + 1 AS gap_start,
       MIN(t2.id) - 1 AS gap_end
FROM orders t1
JOIN orders t2 ON t2.id > t1.id
WHERE NOT EXISTS (SELECT 1 FROM orders t3 WHERE t3.id = t1.id + 1)
```

```
GROUP BY t1.id
HAVING gap_start <= gap_end;
```

Exercise 27: Write a query to detect fraudulent orders (same customer, multiple orders within 1 minute).

```
SELECT
    o1.customer_id, o1.id AS order1, o2.id AS order2,
    o1.created_at AS time1, o2.created_at AS time2,
    TIMESTAMPDIFF(SECOND, o1.created_at, o2.created_at) AS seconds_apart
FROM orders o1
JOIN orders o2 ON o1.customer_id = o2.customer_id
    AND o1.id < o2.id
    AND TIMESTAMPDIFF(SECOND, o1.created_at, o2.created_at) < 60;
```

Exercise 28: Create a materialized summary table and stored procedure to refresh it.

```
CREATE TABLE daily_sales_summary (
    sales_date DATE PRIMARY KEY,
    total_orders INT, total_revenue DECIMAL(15,2),
    avg_order_value DECIMAL(12,2), unique_customers INT,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE PROCEDURE RefreshDailySummary(IN p_date DATE)
BEGIN
    REPLACE INTO daily_sales_summary
    SELECT DATE(created_at), COUNT(*), SUM(total_amount),
           AVG(total_amount), COUNT(DISTINCT customer_id), NOW()
    FROM orders
    WHERE DATE(created_at) = p_date AND status != 'CANCELLED';
END //
DELIMITER ;
```

Exercise 29: Write a query to find the median salary.

```
SELECT AVG(salary) AS median_salary
FROM (
    SELECT salary, ROW_NUMBER() OVER (ORDER BY salary) AS rn,
           COUNT(*) OVER () AS total
    FROM employees
```

```
) sub
WHERE rn IN (FLOOR((total + 1) / 2), CEIL((total + 1) / 2));
```

Exercise 30: Create a stored procedure that generates a complete customer report.

```
DELIMITER //
CREATE PROCEDURE CustomerReport(IN p_customer_id BIGINT)
BEGIN
    -- Customer info
    SELECT id, name, email, created_at FROM customers WHERE id = p_customer_id;

    -- Order summary
    SELECT COUNT(*) AS total_orders, SUM(total_amount) AS total_spent,
           AVG(total_amount) AS avg_order, MAX(created_at) AS last_order
    FROM orders WHERE customer_id = p_customer_id AND status != 'CANCELLED';

    -- Recent orders
    SELECT id, total_amount, status, created_at
    FROM orders WHERE customer_id = p_customer_id
    ORDER BY created_at DESC LIMIT 10;

    -- Top products purchased
    SELECT p.name, SUM(oi.quantity) AS total_qty, SUM(oi.quantity *
oi.unit_price) AS total_spent
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.id
    JOIN products p ON oi.product_id = p.id
    WHERE o.customer_id = p_customer_id
    GROUP BY p.id, p.name ORDER BY total_spent DESC LIMIT 5;
END //
DELIMITER ;
```

End of Guide

Document Version: 1.0 **Last Updated:** March 2026 **Topics Covered:** 17 Parts, 105 Interview Questions, 30 SQL Exercises